Transformer 模型

第 2 章

注意力机制 · 模型结构 · 训练与推理 · 代码实践

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

2.1 **Transformer 概述** — 自注意力机制与 Transformer 的核心价值 — 第 3 页

2.2 **注意力机制** — 序列表征、注意力概念与注意力分数 — 第 7 页

2.3 **Transformer 模型结构** — 输入层、编码器、解码器与输出层 — 第 24 页

2.4 **Transformer 训练和推理** — 并行训练与自回归推理 — 第 76 页

2.5 **关键代码实践** — 缩放点积注意力与多头注意力的 PyTorch 实现 — 第 85 页

2.6 **小结** — 本章要点与位置编码的必要性（附录 2.6.1） — 第 92 页

阅读建议 先建立“按需读取”的直觉（2.2），再读模型结构与训练推理（2.3–2.4），最后对照代码实现（2.5）。

2.1

Transformer 概述

Transformer 以自注意力取代顺序依赖，成为深度学习与大语言模型的基石架构。

本节内容

核心价值

影响范围

自注意力机制

Transformer：深度学习的基石

自 2017 年提出以来，Transformer 已成为深度学习领域的基石性突破和主导力量，重塑了自然语言处理、计算机视觉、语音识别与多模态学习的研究格局。

核心价值 1

强大的表征学习

自注意力直接建模序列中元素之间的关系，学到的表征能够刻画丰富的语义信息。

核心价值 2

架构的统一性

同一架构可以应用于不同模态，文本、图像等数据都能用同样的结构处理。

核心价值 3

卓越的可扩展性

模型规模增大时性能持续提升，因此可以随规模不断扩展。

> **注** 它不仅是大语言模型和多模态大模型的绝对核心，也是推动通用人工智能发展不可或缺的关键技术。

影响：从语言到视觉与多模态

Transformer 先革新了自然语言处理，再跨界进入视觉，并成为多模态模型的统一桥梁。

自然语言处理

BERT：双向编码器

通过双向编码器革新了语言理解。

自然语言处理

GPT 系列：自回归解码器

以自回归解码器和大规模训练，展示了惊人的文本生成与推理能力。

计算机视觉

ViT：视觉新主干

证明纯 Transformer 可以超越 CNN，成为视觉领域的新主干。

多模态

CLIP 与 DALL-E

CLIP 实现图文对齐；DALL-E 根据文本描述生成图像。

文献 Devlin et al., BERT；Radford et al., GPT；Dosovitskiy et al., ViT；Radford et al., CLIP；Ramesh et al., DALL-E

自注意力：模仿人类的注意力集中

注意力的核心优势：针对特定任务，自动学习各元素的重要性并分配相应权重。

类比：高速公路驾驶

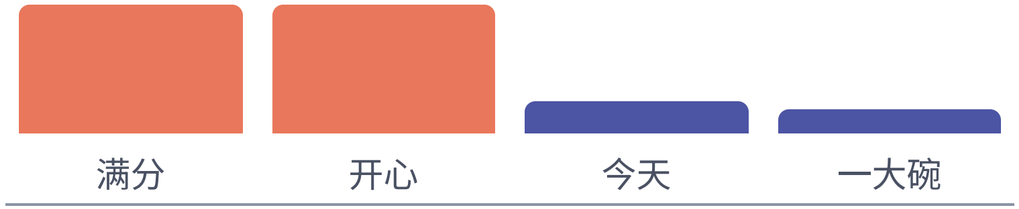
聚焦与安全行驶相关的元素

- 视野中有大量信息：路旁树木、远处风景、广告牌。
- 聚焦前方及侧方车辆、车道标线与交通信号等动态元素。
- 主动降低对路旁静态景物的注意力。

例子：情感分析

关键词获得更高的关注度

“小明今天考试得了满分，回家后开心地吃了一大碗冰淇淋”



柱高为示意，表示相对关注度。

2.2

注意力机制

Transformer 的核心是让每个词元直接读取序列中的其他词元，而不是把历史全部压缩进一个隐状态。

本节内容

2.2.1 序列表征
2.2.2 注意力概念
2.2.3 注意力分数

序列表征：把词序列组合成文本表征

表征是数据在模型中的数学表达，文本表征的目标是把非结构化的自然语言转化为结构化的数值向量。文本本质上是离散的词序列，需要序列建模技术将词的表征组合起来。

直接平均

把序列中所有词的表征取平均作为文本表征。能够全局反映文本语义，但忽略了各词对语义贡献的差异。

真正需要的能力

高效组合序列信息，并区分不同词的重要性。这正是注意力机制要解决的问题。

注：Word2Vec（见 1.4.4 节）只是对词的表征学习，并非对序列表征学习。

词的静态向量、上下文词向量与序列向量

同一个“苹果”，在词表里只有一个向量；放进句子里，每个位置各自得到一个随上下文变化的向量；整个句子还能再压成一个向量。三个层次的形状与含义都不同。

1 · 词的静态向量　Word2Vec

$1 \times d$

从词表查表得到，一个词一个向量，与它出现在哪里无关。词形相同，向量就相同。

2 · 词在序列中的向量

$n \times d$ **的某一行**

经注意力加权融合后，每个位置各自得到一个向量。同一个词出现在不同位置，向量不同。

3 · 整个序列的向量

$1 \times d$

把 n 个位置的向量做池化或取末位，整句压成一个向量，用于分类、检索与句相似度。

层次	形状	随上下文变化	“苹果”在两句话里	来源
词的静态向量	$1 \times d$	否	完全相同	Word2Vec 查表（1.4.4 节）
词在序列中的向量	$n \times d$ 的一行	是	“吃苹果”偏水果，“苹果手机”偏品牌	注意力层输出 $\boldsymbol{Z}$
整个序列的向量	$1 \times d$	—	两句句义不同，句向量也不同	池化，或取末位词元

例　“他吃了一个苹果。”与“苹果发布了新手机。”：静态词向量无法区分两个“苹果”的差别，上下文词向量可以，两句话的序列向量也不同。

从压缩记忆到按需读取

循环网络把历史压缩进隐状态；注意力保留序列中每个时刻的表征，需要时按权重直接读取。

循环神经网络

压缩记忆：逐步整合历史

$\boldsymbol{x}_1$ 词元 → $\boldsymbol{h}_1$ 隐状态 → $\boldsymbol{h}_n$ 固定维度

历史被压缩进固定维度的隐状态，细节难以保留。

注意力机制

按需读取：保留每个表征

$\boldsymbol{x}_1, \cdots, \boldsymbol{x}_n$ 原始表征全部保留 → **权重** 动态计算相关性

按权重对历史各时刻的表征加权组合。

例 读第 9 章时需要参考第 1 章：可依靠大脑的压缩记忆（类似 RNN），也可直接翻到第 1 章定位（类似注意力）。注意力的检索避免了信息遗失，代价是存储与计算复杂度上升。

从文本到词元：查表得到编号序列

文本
原始字符串

→

词元
分词后的基本离散单元

→

词元编号
词表中的唯一编号

→

词向量
词嵌入表示（见 1.4.3 节）

- 词元是文本划分后的基本离散单元；字符级分词器中，每个字符即一个词元（见 1.4.2 节）。
- 独立的词向量只能编码词元本身的基础语义；理解词元在序列中的实际语义，必须考虑上下文。
- 传统的规则方法难以覆盖复杂多变的语言模式。注意力机制以可学习、数据驱动的方式，直接建模并量化任意两个词元之间的关联程度。

中文例子：主谓宾关系跨越了修饰语

“喜欢深度学习的学生提交了一份精彩的报告”——句子骨架只有“学生—提交—报告”三块，其余都是修饰语；而修饰语里的“喜欢”管的是“深度学习”，不是“学生”。

词元	词性	句法角色	与谁相关
学生	名词	主语	提交（主谓），相隔 3 个词元
提交	动词	谓语	学生（主谓）、报告（动宾）
报告	名词	宾语	提交（动宾）
喜欢	动词	定语从句谓语	深度学习（动宾），不是“学生”
精彩	形容词	定语	报告（定中）
的	助词	结构助词	标记“的”字结构，本身不承载语义

表中每一行的“与谁相关”就是一条需要建模的边；有些边相邻，有些隔了三四个词元。

> **关键** “学生”要读到“提交”，跨度 3 个词元；“喜欢”要读到“深度学习”而不是紧邻的“学生”。这样的边在句子里有成百上千条，无法逐条写规则，只能交给可学习的权重。

传统做法的链条

一串模型，逐级传误差

- 分词：中文没有空格，先要切词，再处理歧义与未登录词。
- 词性标注：同形词常有多类，需要 HMM、CRF 一类模型。
- 句法分析：依存或短语结构语法，规则写不完，还要用转移式、图式模型训练。
- 前一步错了，后一步全错；换一种语言，整条链要重做。

英文例子：倒装与定语从句

“Never have I seen a model that captures such long-range dependencies so well.”——否定副词提前引起倒装，从句谓语又要与先行词一致：两处关系都不在相邻词元之间。

现象	关系	相隔	说明
倒装	have → I	相邻	Never 提到句首，助动词 have 移到主语 I 之前，主谓一致按 I 判断。
倒装	seen → I	1 个词元	谓语 seen 的逻辑主语是 I，中间隔着助动词。
定语从句	that → model	相邻	关系代词 that 指代先行词 model，而不是它前面的冠词 a。
定语从句	captures → model	4 个词元	从句谓语与先行词 model 单数一致，不能看紧邻的复数 dependencies。
动宾	captures → dependencies	5 个词元	从句谓语与宾语之间隔着定语 long-range。

传统做法

语法知识与工具各管一段

先要有一套短语结构或依存语法标注，再训练转移式、图式解析器；主谓一致、从句边界这类现象还要额外写规则与特征。

注意力

直接学“谁与谁相关”

不必先切分句子成分：每个位置对全部位置算一行分数，权重大的位置就是它读取的对象。语法关系成为可学习的权重。

结论 语言的结构性体现在词与词的关系上。把“关系”做成可学习的量，就不必为每种语言、每种现象各写一套分析器。

词元之间的注意力关系（示例 1）

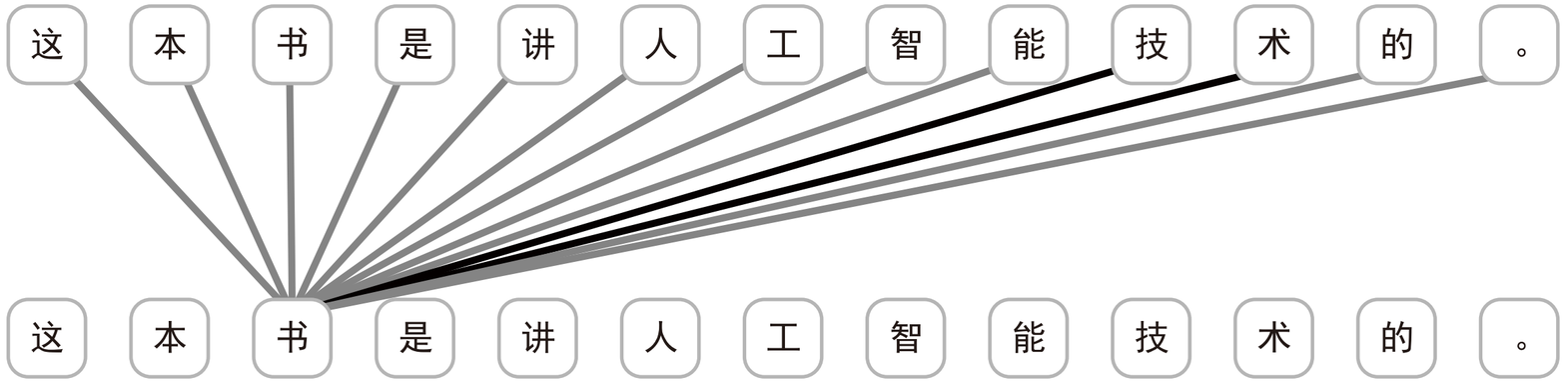


图 2.1　词元在序列中的注意力关系（示例 1）

- 连线越粗，表示两个词元之间的注意力权重越大。
- 句子“这本书是讲人工智能技术的。”中，词元“书”与内容词元“技术”可能有较强关联，与句末标点“。”的关联则相对较弱（并非完全无关）。

图源：《大语言模型：训练、精调与对齐》图 2.1

多头视角：同一句子的不同关系



图 2.2　词元在序列中的注意力关系（示例 2）



图 2.3　词元在序列中的注意力关系（示例 3）

多头注意力让模型从不同“角度”关注词元间的不同关系，更有效地捕捉文本内部复杂的语义关联。

图 2.2：词元“这”的注意力权重高度集中在指代对象“书”上。未显示的连线不代表绝对无关，仅表示关联度低于可视化阈值。

图 2.3：对于词元“书”，一个注意力头聚焦于与谓语动词“是”的语法关系（强关联），同时关注与宾语“技术”的语义联系。

图源：《大语言模型：训练、精调与对齐》图 2.2、图 2.3

注意力的作用：上下文感知的表示

学习权重
词元之间的关联程度

→

加权融合
对词向量加权融合

→

动态表示
每个词元的上下文感知语义

→

序列表示
序列级别的语义表示

- 通过调整注意力权重，可以灵活改变每个词元的语义表示，聚焦于对当前任务更重要的上下文信息。
- 注意力的核心是系统性地学习序列中词元之间的关联程度（注意力权重），并利用它们对词向量加权融合。
- 具体的注意力计算（缩放点积注意力）详见 2.3.4 节。

注意力分数：点积度量词元相关性

注意力机制需要度量词元之间的相关性，常用方法之一是计算词向量之间的点积，作为注意力分数。

$$\boldsymbol{x}_i \in \mathbb{R}^d$$

$$\boldsymbol{X} = [\boldsymbol{x}_1; \boldsymbol{x}_2; \cdots; \boldsymbol{x}_n] \in \mathbb{R}^{n \times d}$$

$\boldsymbol{x}_i$ 为第 i 个词元的词向量，d 为向量维度；n 个词元的词向量按行拼接，得到词矩阵 $\boldsymbol{X}$。

1 · 词与词

$\boldsymbol{s}_{ij}$

两个词元 i、j 之间的标量分数，见图 2.4(a)。

2 · 词与序列

$\boldsymbol{S}_i$

一个词元与全部 n 个词元的分数行向量，见图 2.4(b)。

3 · 序列与序列

$\boldsymbol{S}$

所有词元对之间的 $n \times n$ 自注意力分数矩阵，见图 2.4(c)。

词元之间的注意力分数 s_{ij}

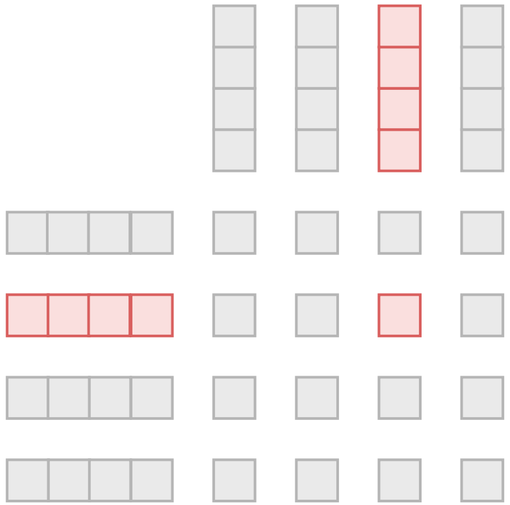


图 2.4 (a)　词与词

$$
\begin{aligned}
s_{ij} &= \boldsymbol{x}_i \cdot \boldsymbol{x}_j \\
&= \sum_{k=1}^{d} x_{ik}\, x_{jk} \\
&= \boldsymbol{x}_i \boldsymbol{x}_j^{T}
\end{aligned}
\tag{2-1}
$$

点积的定义；按分量展开，x_{ik} 为 $\boldsymbol{x}_i$ 的第 k 个分量。$\boldsymbol{x}_i$ 为 $1 \times d$ 行向量，$\boldsymbol{x}_j^T$ 为 $d \times 1$ 列向量，二者相乘得到 1×1 标量，即式 (2-1)。

$s_{ij} \in \mathbb{R}$ 为标量，s_{ij} 越大，词元 i、j 的相关性越强。完整的注意力分数还包含缩放（Scaled），以控制分数的方差。①

① 度量向量相关性的方法有多种，点积是注意力机制中的标准实现。

图源：《大语言模型：训练、精调与对齐》图 2.4；依据式 (2-1)

单个词元的注意力分数向量 $\boldsymbol{S}_i$

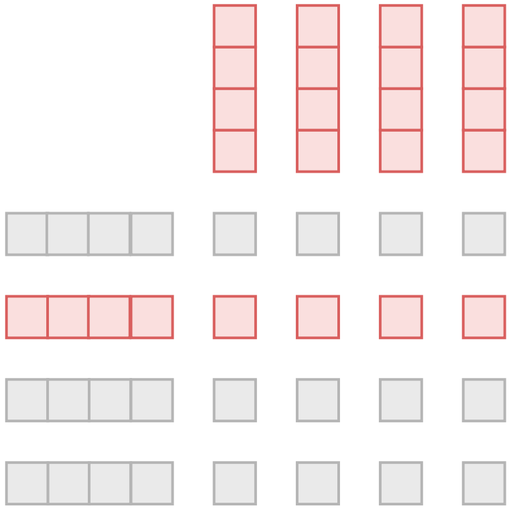


图 2.4 (b)　词与序列

$$\boldsymbol{S}_i = \boldsymbol{x}_i \boldsymbol{X}^T \tag{2-2}$$

$$\boldsymbol{X}^T = \left[\boldsymbol{x}_1^T, \cdots, \boldsymbol{x}_n^T\right]$$

$$\boldsymbol{S}_i = \boldsymbol{x}_i \left[\boldsymbol{x}_1^T, \cdots, \boldsymbol{x}_n^T\right] = \left[\boldsymbol{x}_i \boldsymbol{x}_1^T, \cdots, \boldsymbol{x}_i \boldsymbol{x}_n^T\right]$$

$$= \left[\boldsymbol{s}_{i1}, \cdots, \boldsymbol{s}_{in}\right]$$

式 (2-2) 给出 $\boldsymbol{S}_i$；$\boldsymbol{X}$ 按行堆叠，转置后 $\boldsymbol{X}^T$ 按列拼接。矩阵乘法按列分块后，每一项都是式 (2-1) 中的 $\boldsymbol{s}_{ij}$。维度为 $(1 \times d)(d \times n) = 1 \times n$。

图源：《大语言模型：训练、精调与对齐》图 2.4；依据式 (2-2)

完整的自注意力分数矩阵 S

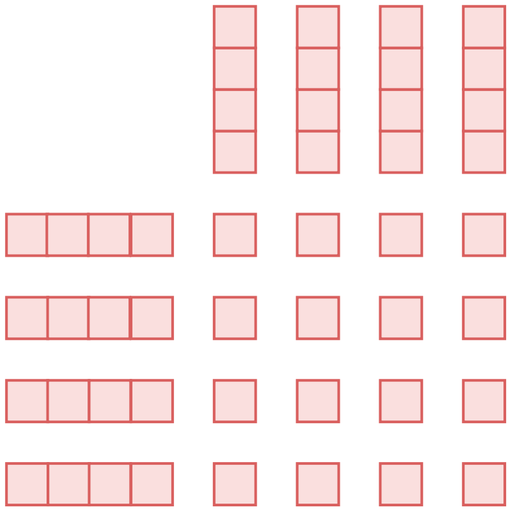


图 2.4 (c)　序列与序列

$$\boldsymbol{S} = \begin{bmatrix} \boldsymbol{S}_1 \\ \vdots \\ \boldsymbol{S}_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1 \boldsymbol{X}^T \\ \vdots \\ \boldsymbol{x}_n \boldsymbol{X}^T \end{bmatrix} = \begin{bmatrix} \boldsymbol{x}_1 \\ \vdots \\ \boldsymbol{x}_n \end{bmatrix} \boldsymbol{X}^T = \boldsymbol{X}\boldsymbol{X}^T \tag{2-3}$$

第一步把各 $\boldsymbol{S}_i$ 按行堆叠，并代入式 (2-2)；第二步把公共因子 $\boldsymbol{X}^T$ 提到右侧，左侧堆叠的行向量即 $\boldsymbol{X}$，得到式 (2-3)。维度为 $n \times d$ 乘 $d \times n$，得 $n \times n$。

矩阵 $\boldsymbol{S}$ 的第 i 行第 j 列元素即 s_{ij}，包含序列中所有词元对之间的自注意力分数。

图源：《大语言模型：训练、精调与对齐》图 2.4；依据式 (2-3)

多头注意力：并行计算多组注意力关系

阅读第 10 章情节时，若注意力仅与第 1 章存在强关联，就可能忽视与第 6 章等其他章节的重要联系。多头注意力（MHA）通过并行计算多个独立的注意力关系，捕获更丰富的上下文信息。

单头

只有一组注意力关系

一个分数矩阵只能呈现一种依赖模式，容易遗漏其他重要联系。

多头

多组独立的注意力关系

将词向量沿特征维切分为 h 个头，每个头各自计算注意力分数，再分别读取。

$\boldsymbol{X} \in \mathbb{R}^{n \times d}$

原词向量矩阵

→

$\boldsymbol{X}^{(1)}, \cdots, \boldsymbol{X}^{(h)}$

沿特征维切分为 h 份

→

$\boldsymbol{S}^{(1)}, \cdots, \boldsymbol{S}^{(h)}$

每个头各自的分数矩阵

多头注意力关系（$h = 2$）

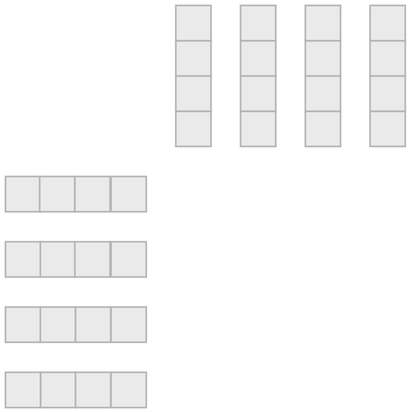


图 2.5 (a)　原始维度

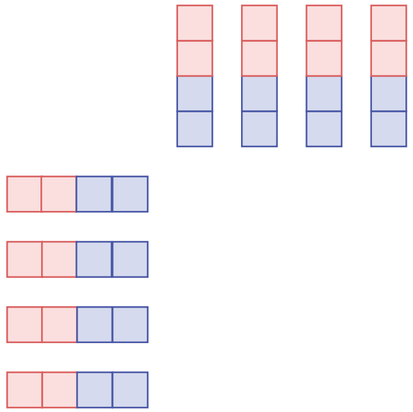


图 2.5 (b)　多头维度

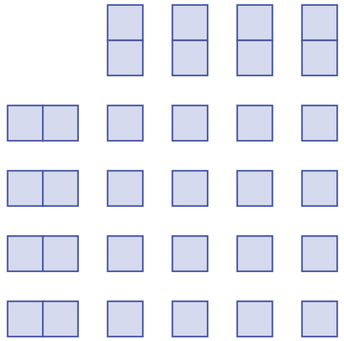


图 2.5 (c)　注意力头 1

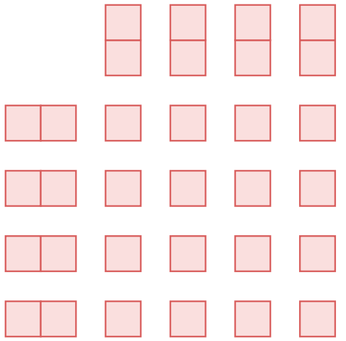


图 2.5 (d)　注意力头 2

- (a) 原始的输入序列向量表示；(b) 将每个词向量沿特征维切分为两个头（$h = 2$）。
- (c)、(d) 分别示意两个头独立计算出的注意力权重矩阵。
- 不同头关注不同依赖：一个头关注章节间的顺序依赖（第 10 章与第 1 章），另一个头关注主题联系（第 10 章与第 6 章）。

图源：《大语言模型：训练、精调与对齐》图 2.5

各头的注意力分数矩阵 $\boldsymbol{S}^{(k)}$

$$\boldsymbol{X}=\left[\boldsymbol{X}^{(1)},\boldsymbol{X}^{(2)},\cdots,\boldsymbol{X}^{(h)}\right],\quad \boldsymbol{X}^{(k)}\in\mathbb{R}^{n\times d'}$$

$$d=h\cdot d'\ \Rightarrow\ d'=d/h$$

$$\boldsymbol{S}^{(k)}=\boldsymbol{X}^{(k)}\left(\boldsymbol{X}^{(k)}\right)^{T}\tag{2-4}$$

1 · 切分

把每个词向量沿特征维切为 h 份，每份 $d'=d/h$ 维。

2 · 计算

式 (2-3) 中的 $\boldsymbol{X}$ 换成 $\boldsymbol{X}^{(k)}$，维度 $(n\times d')(d'\times n)=n\times n$。

3 · 输出

得到 h 个 $n\times n$ 的分数矩阵，各头学习不同的依赖模式。

依据式 (2-4)，参见第 2.3.4 节（多头注意力机制）

2.3

Transformer 模型结构

先看翻译数据与训练目标，再逐层拆解输入层、编码器、解码器与输出层，并看清注意力层在其中的位置。

本节内容

2.3.1 机器翻译数据

2.3.2 Transformer 模型架构和训练目标

2.3.3 输入层

2.3.4 编码器

2.3.5 解码器

2.3.6 输出层

文本先被切分为词元序列

分词器把文本转换为词元序列 $\boldsymbol{w} = (w_1, w_2, \cdots, w_n)$，每个词元都属于词表 $\mathcal{V}$。序列两端需要加入三类特殊词元。

特殊词元

<SOS>　句首

标记句子开始。解码器看到它时启动生成。

特殊词元

<EOS>　句尾

标记句子结束。解码器输出它时终止生成。

特殊词元

<PAD>　填充

把批量内长度不一的序列补齐到相同长度，便于批量训练。

这些人工定义的词元划定了变长序列的始与终，是后续生成与批量训练的基础。

双语平行语料：输入与标签

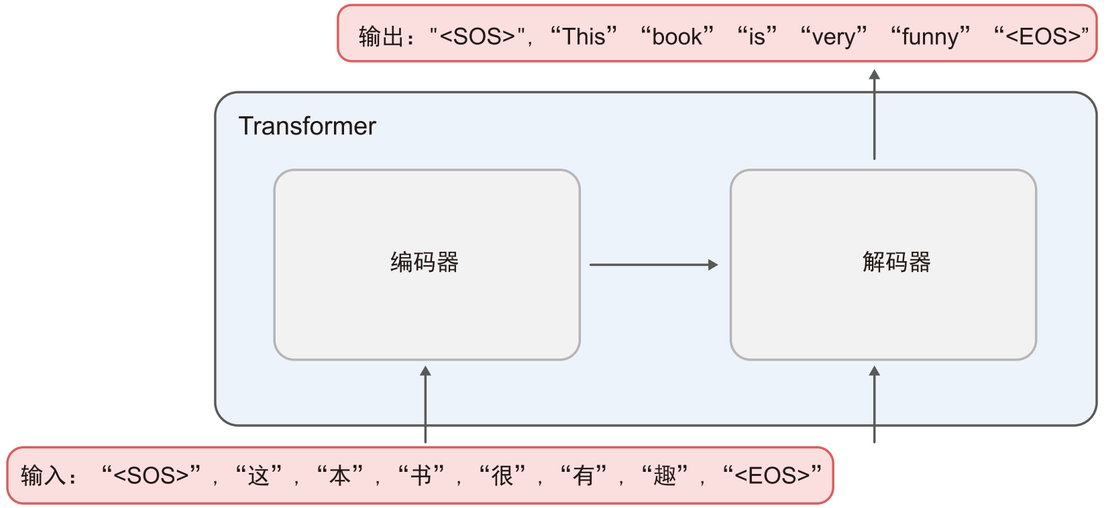


图 2.6　Transformer 模型输入输出是双语成对的数据

- 翻译数据由不同语言的平行文本对组成：源语言 $\boldsymbol{x} = (x_1, \cdots, x_m)$ 作为输入，目标语言 $\boldsymbol{y} = (y_1, \cdots, y_n)$ 作为标签。
- 两种语言使用独立的分词器与词表。图 2.6 只展示输入与输出，未包含解码器输入细节，见图 2.7。

图源 《大语言模型：训练、精调与对齐》图 2.6

编码器-解码器结构与自回归预测

早期翻译模型

编码器把输入压缩为一个上下文向量；解码器据此逐步生成译文。

Transformer

源序列 $\boldsymbol{x}$ 经编码器得到隐状态矩阵 $\boldsymbol{Z} \in \mathbb{R}^{m \times d}$；解码器利用 $\boldsymbol{Z}$ 与已生成的 $\boldsymbol{y}_{\leqslant t}$ 预测下一个词元 y_{t+1}。

$$p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)$$

解码器是自回归模型：时刻 t 的预测依赖源序列 $\boldsymbol{x}^{(k)}$ 与已生成的前缀 $\boldsymbol{y}_{\leqslant t}^{(k)} = (y_1^{(k)}, \cdots, y_t^{(k)})$。

数据集 $\mathcal{D}$ 含 K 个样本对 $\left(\boldsymbol{x}^{(k)}, \boldsymbol{y}^{(k)}\right)$，$\boldsymbol{x}^{(k)}$ 为源语言词元序列，$\boldsymbol{y}^{(k)}$ 为目标语言词元序列。

架构：输入层、编码器、解码器与输出层

输入层

词嵌入 + 位置编码（2.3.3）

→

编码器

多层编码块（2.3.4）

→

解码器

多层解码块（2.3.5）

→

输出层

线性层 + Softmax（2.3.6）

整体共有 3 种注意力层，它们都是后面各层的核心部件：

注意力 ①

编码器自注意力

源序列内部的词元互相读取，见 2.3.4。

注意力 ②

解码器掩码自注意力

目标位置只能读取它及之前的词元，见 2.3.5。

注意力 ③

编码-解码注意力

解码器读取编码器输出 $\boldsymbol{Z}$，把源语言信息送入解码，见 2.3.5。

三类注意力层都由 2.2 节的缩放点积注意力与多头机制构成，区别只在于 $\boldsymbol{Q}$、$\boldsymbol{K}$、$\boldsymbol{V}$ 的来源与所用掩码。

整体框架与三种注意力的位置

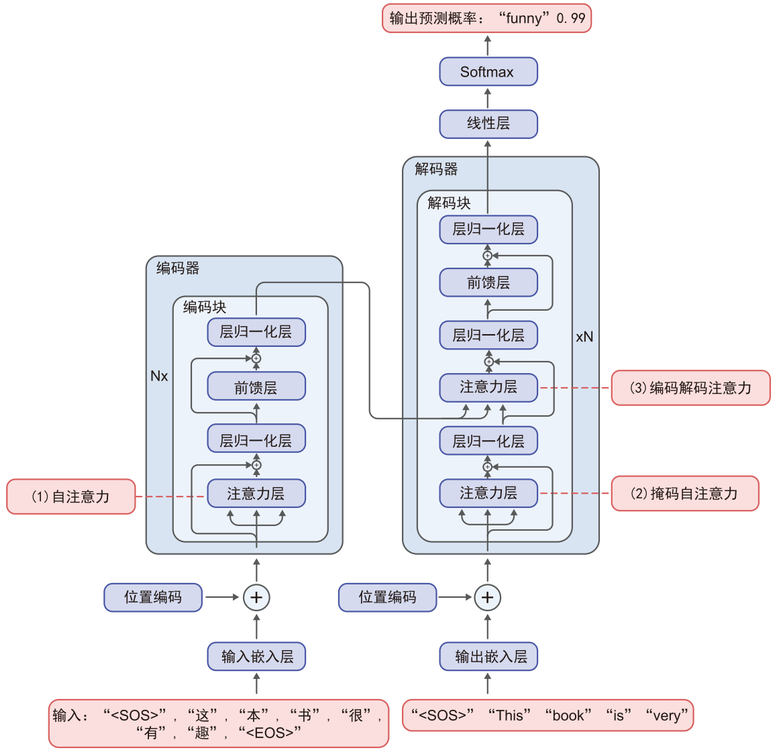


图 2.7　Transformer 网络架构图

- ①　编码器自注意力（2.3.4）：源序列内部互相读取。
- ②　解码器掩码自注意力（2.3.5）：目标序列只看已生成的部分。
- ③　编码-解码注意力（2.3.5）：目标位置读取编码器输出 $\boldsymbol{Z}$。
- 三者都由 2.2 节的缩放点积注意力与多头机制构成。

左侧为编码器堆叠，右侧为解码器堆叠；两侧之间只有编码-解码注意力的箭头，源语言信息由此进入解码。

图源　《大语言模型：训练、精调与对齐》图 2.7

训练样本：用已知前缀预测下一个词元

源句“<SOS>这本书很有趣<EOS>”输入编码器。解码器输入为目标语言前缀，监督标签为下一个词元。

<SOS> This book is very

解码器输入（已知前缀）

→

funny

预测标签（下一个词元）

- 输出层给出词表中所有词元的概率分布，而非直接给出某个词元。
- 训练目标是最大化标签 “funny” 对应的概率，即监督学习。
- 目标序列含多个词元时，每个位置都构成一个预测任务。

单个样本的损失：负对数似然

$$
\begin{aligned}
&\max_{\theta} \prod_{t=1}^{T_k} p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right) \\
&= \max_{\theta} \sum_{t=1}^{T_k} \log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right) \\
&= \min_{\theta} \left[-\sum_{t=1}^{T_k} \log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right] \\
\mathcal{L}^{(k)}(\theta) &= -\frac{1}{T_k} \sum_{t=1}^{T_k} \left[\log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right]
\end{aligned}
$$

式 (2-5)：依次为最大化似然、取对数、取负号，再按长度 T_k 取平均。

注 T_k 为第 k 条数据的词元序列长度；$p_\theta(\cdot)$ 为参数 θ 下的预测分布。

数据集的总损失

$$
\begin{aligned}
\mathcal{L}(\theta) &= \frac{1}{K}\sum_{k=1}^{K}\mathcal{L}^{(k)}(\theta) \\
&= \frac{1}{K}\sum_{k=1}^{K}\left(-\frac{1}{T_k}\sum_{t=1}^{T_k}\log p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right) \\
&= -\frac{1}{K}\sum_{k=1}^{K}\frac{1}{T_k}\sum_{t=1}^{T_k}\log\left(p_\theta\left(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}\right)\right)
\end{aligned}
\tag{2-6}
$$

第一步对 K 个样本级损失取平均；第二步代入式 (2-5)；第三步去掉外层括号，即式 (2-6)。

最小化 $\mathcal{L}(\theta)$ 等价于最大化预测目标序列的概率；它是标准交叉熵损失在自回归序列建模中的形式。

输入层：词嵌入与位置编码

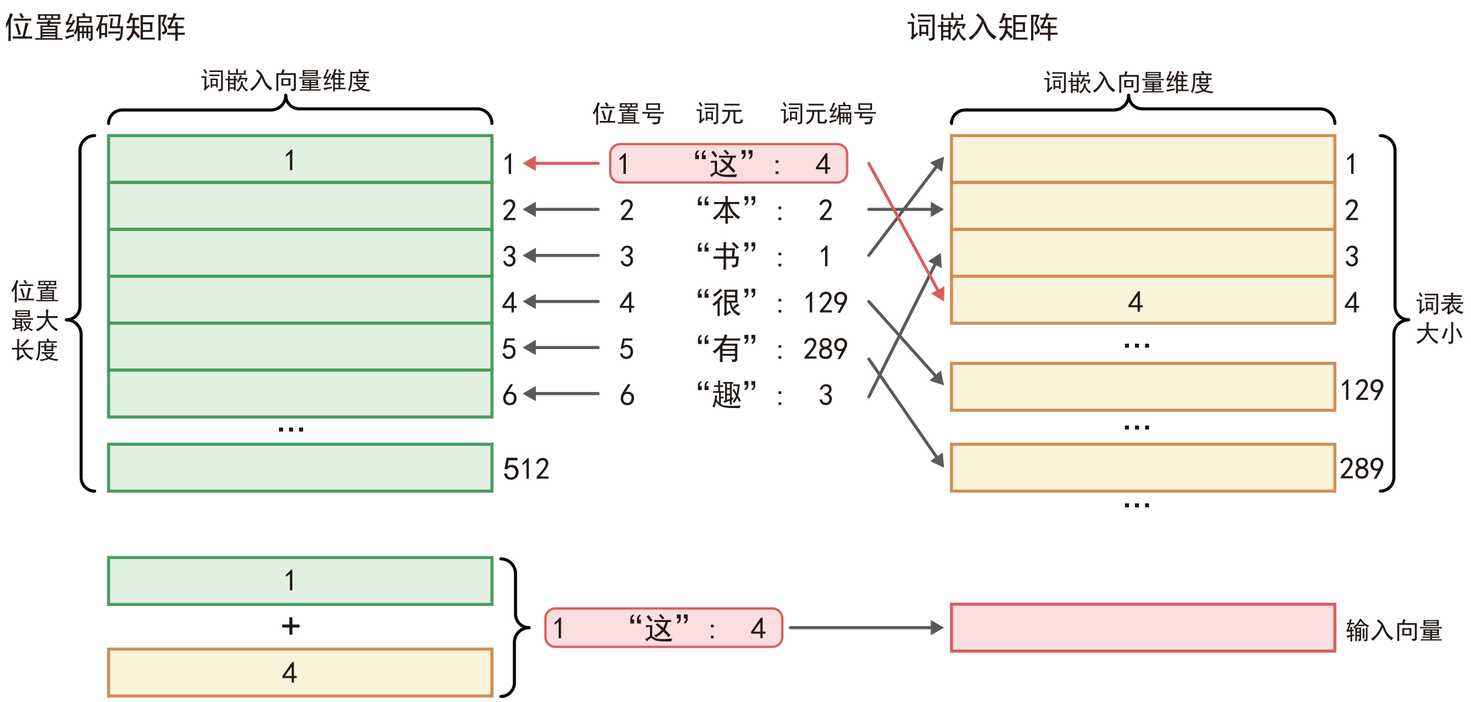


图 2.8　Transformer输入层示例

- 以首位词元“这”（词元编号 4）为例。
- 嵌入层从嵌入参数矩阵中查得它的词嵌入向量。
- 位置编码层从位置编码矩阵中查得该位置的向量。
- 两者相加，得到融合位置信息的词向量。

图源 《大语言模型：训练、精调与对齐》图 2.8

嵌入层：按词元编号查表

$$\boldsymbol{E} \in \mathbb{R}^{|\mathcal{V}| \times d}$$

可学习的词嵌入参数矩阵，每一行对应词表中的一个词元。

$$\boldsymbol{x}_i = \boldsymbol{E}_{x_i} \in \mathbb{R}^{1 \times d}$$

取出第 x_i 行，即词元编号 x_i 对应的词嵌入向量。

$$\boldsymbol{X} = \left[\boldsymbol{x}_1; \boldsymbol{x}_2; \cdots; \boldsymbol{x}_n\right] \in \mathbb{R}^{n \times d}$$

按行拼接得到输入的词嵌入矩阵。

图 2.9 中 6 个词元即被表示为 $\boldsymbol{X} \in \mathbb{R}^{6 \times d}$。

嵌入矩阵的查表过程与初始化

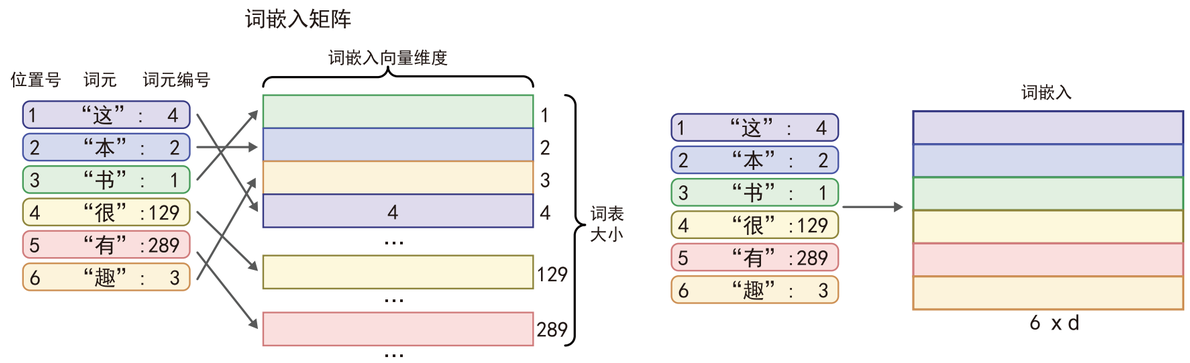


图 2.9　词元编号序列在嵌入层参数矩阵逐行取向量组成词嵌入输入矩阵

初始化

正态分布

$\boldsymbol{E}$ 通常用均值为 0、标准差 $\sigma = 0.02$ 的正态分布初始化，并随训练通过反向传播学习。

填充符

<PAD> 为零向量

填充符只用于统一长度，无实际语义，其词嵌入通常设为零向量。

独立嵌入

编码器与解码器分开

两侧处理不同语言、拥有各自的词表，因此使用独立的输入嵌入层。

图源 《大语言模型：训练、精调与对齐》图 2.9

为什么必须显式加入位置信息

语序的改变可能改变语义，模型需要同时感知词元的绝对位置与词元之间的相对位置。

问题

Transformer 不按顺序递归

RNN 类模型按时间步递归处理序列，天然携带顺序；Transformer 不能这样做，需要把位置信息显式加入词嵌入。

约束

自注意力对顺序不敏感

位置信息弥补了注意力机制本身对序列顺序不敏感的缺陷，使模型能理解序列特性。

要求 1

唯一性

每个位置的编码向量都不同。

要求 2

稠密性

所有维度都包含有效信息，避免固定为 0 的冗余维度。

要求 3

多频率

不同维度捕获不同变化频率的位置模式。

二进制位置编码：唯一，但维度利用率低

表 2.1　二进制位置编码向量

位置号	β 转换	二进制编码向量
0	β_0	[0,0,0,0,0,0,0,0]
1	β_1	[0,0,0,0,0,0,0,1]
2	β_2	[0,0,0,0,0,0,1,0]
3	β_3	[0,0,0,0,0,0,1,1]
4	β_4	[0,0,0,0,0,1,0,0]
5	β_5	[0,0,0,0,0,1,0,1]
6	β_6	[0,0,0,0,0,1,1,0]
7	β_7	[0,0,0,0,0,1,1,1]
⋮	⋮	⋮
15	β_{15}	[0,0,0,0,1,1,1,1]
64	β_{64}	[0,1,0,0,0,0,0,0]

- 位置编码矩阵 $\mathbf{PE} \in \mathbb{R}^{L_{\max} \times d}$，通常预先算好、训练时不学习。
- 每个位置号对应唯一的 8 位编码。
- 最右端为最低位，变化最快、周期最短；最左端为最高位，变化最慢、周期最长。
- 当 $d = 512$、$L_{\max} = 1024$ 时，仅需低 $\lceil \log_2 1024 \rceil = 10$ 位即可唯一表示所有位置，高 502 位始终为 0，成为冗余维度，模型难以利用。

正弦位置编码（式 (2-7)）

$$\begin{aligned}\mathbf{PE}_m &= \left[\sin(m\theta_1), \cos(m\theta_1), \sin(m\theta_2), \cos(m\theta_2), \cdots, \sin(m\theta_t), \cos(m\theta_t)\right] \\ &= \left[\boldsymbol{p}_{m,1}, \boldsymbol{p}_{m,2}, \cdots, \boldsymbol{p}_{m,t}\right], \qquad \boldsymbol{p}_{m,i} = \left(\sin(m\theta_i),\ \cos(m\theta_i)\right)\end{aligned}$$

式 (2-7)：每两维为一组，第 i 组由同一角度 $m\theta_i$ 的正弦与余弦组成，再按组拼接。

$$\theta_i = \frac{1}{b^{2(i-1)/d}}$$

m

位置号

词元在序列中的位置，从 0 开始。

$t = d/2$

角度组数

d 维词向量每两维一组，共 t 组。

θ_i

第 i 组角度

$\theta_1 = 1$，随 i 增大而减小。

b

基数 $b = 10\,000$

使 θ_i 落在合适的取值范围内。

角度随维度衰减：由高频到低频

$\theta_1 = \dfrac{1}{b^0} = 1$，$\theta_t = \dfrac{1}{b^{(d-2)/d}}$，即 $\dfrac{1}{b^{(d-2)/d}} \leqslant \theta_i \leqslant 1$。随 i 增大，$\sin(m\theta_i)$ 与 $\cos(m\theta_i)$ 的频率降低，高维变化更平缓。

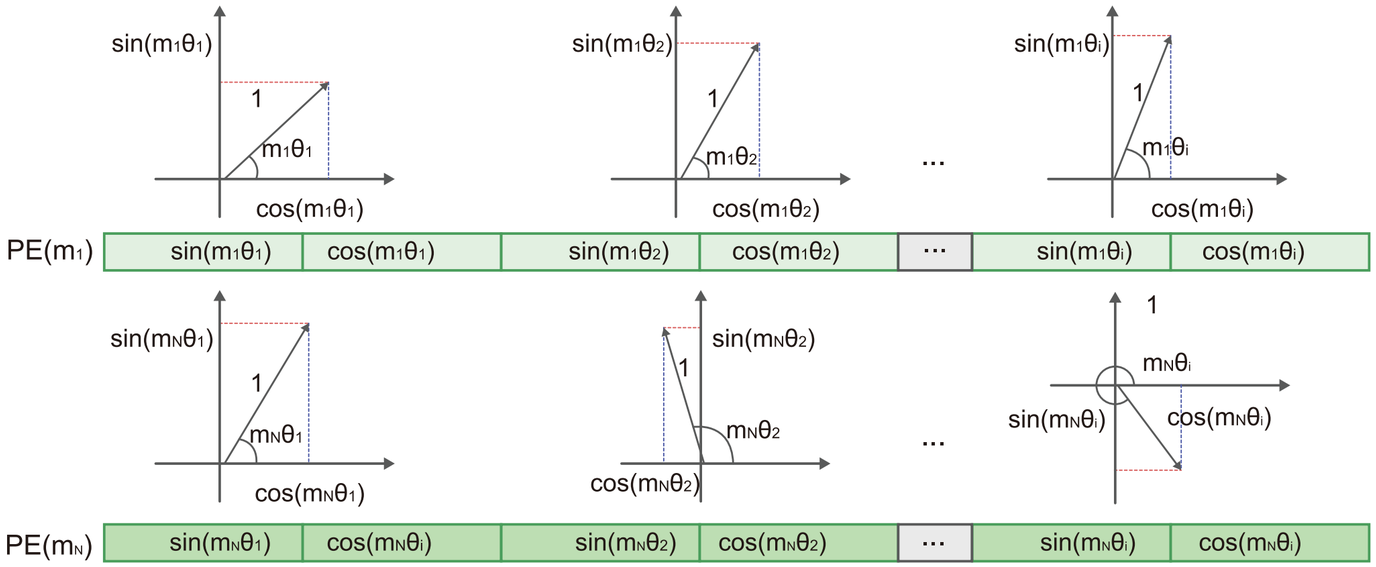


图 2.10　位置编码向量由不同角度的正弦和余弦构成

图源 《大语言模型：训练、精调与对齐》图 2.10

输入表示：词嵌入与位置编码相加（式 (2-8)）

$$\boldsymbol{x}_i' = \boldsymbol{x}_i + \mathrm{PE}_i$$

式 (2-8)：$\boldsymbol{x}_i$ 与 PE_i 同为 $1 \times d$，逐元素相加后仍为 $1 \times d$。

$$\boldsymbol{X}' = [\boldsymbol{x}_1 + \mathrm{PE}_1;\ \cdots;\ \boldsymbol{x}_n + \mathrm{PE}_n] = \boldsymbol{X} + [\mathrm{PE}_1;\ \cdots;\ \mathrm{PE}_n] \in \mathbb{R}^{n \times d}$$

按行堆叠得到输入矩阵，形状保持 $n \times d$。

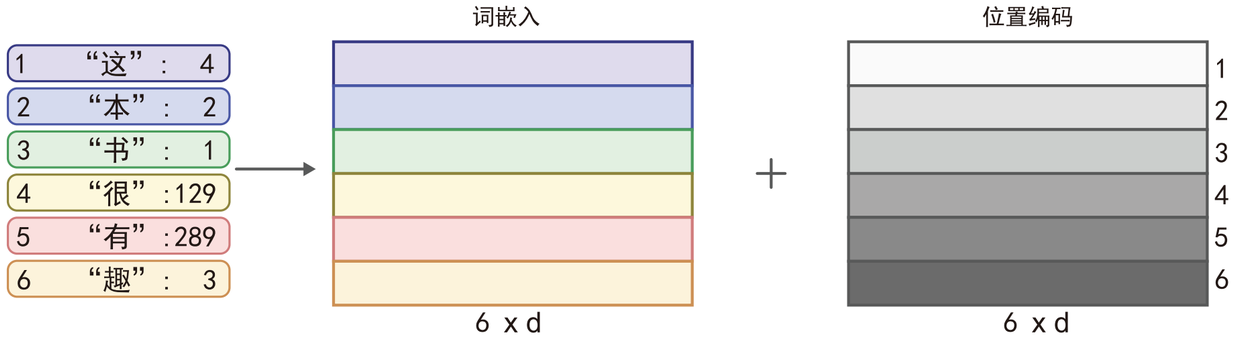


图 2.11　Transformer 输入层处理示例

图源 《大语言模型：训练、精调与对齐》图 2.11

位置信息的融合方式与 Dropout

方式	做法
逐元素操作	每个元素乘以位置相关的标量（缩放）或加上位置相关的偏置（平移）
向量加法（最常用）	词嵌入与位置编码相加，即式 (2-8)；可视为引入位置相关的偏置，能加速收敛
矩阵变换	位置编码通过矩阵乘法作用于词向量，如旋转位置编码 RoPE（详见第 4 章）
向量拼接	位置编码与词嵌入拼接，但会增加特征维度
学习式编码	用可学习参数向量表示位置信息

训练时

输入层的 Dropout

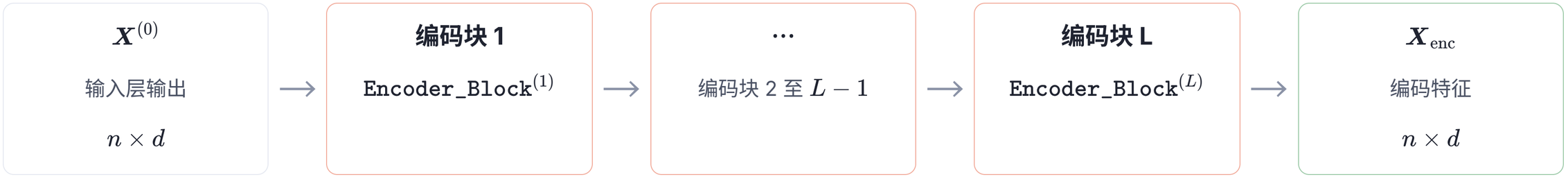
为提升鲁棒性，可对 $\boldsymbol{x}_i'$ 以概率 p 随机将部分维度置零。随着特征维度增加，当前主流大模型往往省略输入层的 Dropout。

编码器：多个编码块堆叠

编码器对输入词元序列逐块变换，最终为每个词元输出编码特征 $\boldsymbol{X}_{\text{enc}}$，供解码器的各个解码块读取。

$$\boldsymbol{X}^{(l)} = \texttt{Encoder_Block}^{(l)}\left(\boldsymbol{X}^{(l-1)}\right) \tag{2-9}$$

$\boldsymbol{X}^{(l)}$ 为第 l 个编码块的输出，维度与输入 $\boldsymbol{X}^{(l-1)}$ 一致；$\boldsymbol{X}^{(0)}$ 为输入层的输出，即首个编码块的输入。



- 每个编码块的输入与输出维度相同，因此块可以一层一层堆叠。
- $\boldsymbol{X}_{\text{enc}} \in \mathbb{R}^{n \times d}$ 会被送入解码器的每个解码块（见 2.3.5）。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-9)）

编码块的组成

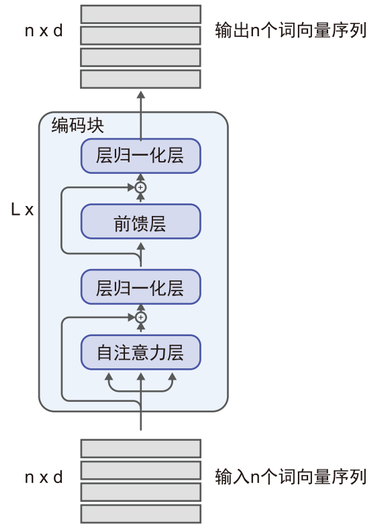


图 2.12　编码块的输入输出维度一致

残差连接：子层输出与子层输入相加

子层 1

自注意力层

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 均来自本块输入；掩码 $\boldsymbol{M}$ 屏蔽 <PAD> 等不参与计算的词元。

归一化

层归一化（LN）

`Layer_Norm_1` 在自注意力层之后，`Layer_Norm_2` 在前馈层之后。

子层 2

前馈层

先升维做非线性激活，再降维还原为 d 维。

图 《大语言模型：训练、精调与对齐》图 2.12

编码块的计算：两个子层，各接残差

$$\boldsymbol{X}_{\text{attn}} = \texttt{Multi_Head_Attention}\left(\boldsymbol{X}^{(l-1)}, \boldsymbol{M}\right) \tag{2-10}$$

$$\boldsymbol{X}_{\text{res1}} = \boldsymbol{X}_{\text{attn}} + \boldsymbol{X}^{(l-1)} \tag{2-11}$$

$$\boldsymbol{X}_{\text{ln1}} = \texttt{Layer_Norm_1}\left(\boldsymbol{X}_{\text{res1}}\right) \tag{2-12}$$

$$\boldsymbol{X}_{\text{ffn}} = \texttt{Feed_Forward}\left(\boldsymbol{X}_{\text{ln1}}\right) \tag{2-13}$$

$$\boldsymbol{X}_{\text{res2}} = \boldsymbol{X}_{\text{ffn}} + \boldsymbol{X}_{\text{ln1}} \tag{2-14}$$

$$\boldsymbol{X}^{(l)} = \texttt{Layer_Norm_2}\left(\boldsymbol{X}_{\text{res2}}\right) \tag{2-15}$$

式 (2-10)–(2-12) 为注意力子层，式 (2-13)–(2-15) 为前馈子层。每个子层“先残差相加，再归一化”：$\boldsymbol{X}_{\text{res1}}$、$\boldsymbol{X}_{\text{res2}}$ 是残差相加后的中间结果。

注意力子层

$\boldsymbol{X}^{(l-1)}$ 经多头注意力得到 $\boldsymbol{X}_{\text{attn}}$，与 $\boldsymbol{X}^{(l-1)}$ 相加后再归一化，得到 $\boldsymbol{X}_{\text{ln1}}$。

前馈子层

$\boldsymbol{X}_{\text{ln1}}$ 经前馈层得到 $\boldsymbol{X}_{\text{ffn}}$，与 $\boldsymbol{X}_{\text{ln1}}$ 相加后再归一化，得到 $\boldsymbol{X}^{(l)}$；维度始终不变。

顺序 残差相加在归一化之前：主干路径上保留一条不做归一化的直通支路，梯度可以无衰减地回传，这正是深层堆叠能训练起来的原因。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-10)–(2-15)）

注意力：对词元序列做加权组合

注意力是序列特征的组合机制：每个词元的新表示，由序列中各词元的表示加权得到，能够捕捉长距离依赖。下面的分数定义与 2.2.3 节一致。

$$\boldsymbol{X}' = \texttt{Multi_Head_Attention}(\boldsymbol{X}, \boldsymbol{M}) \tag{2-16}$$

$\boldsymbol{X}, \boldsymbol{X}' \in \mathbb{R}^{n\times d}$，$n$ 为词元序列长度，d 为特征维度；$\boldsymbol{M}$ 为掩码矩阵。

$$s_{ij} = \boldsymbol{x}_i \boldsymbol{x}_j^T \tag{2-17}$$

任意两个词元特征 $\boldsymbol{x}_i, \boldsymbol{x}_j \in \mathbb{R}^{1\times d}$ 之间的注意力分数，度量词元 i 与词元 j 的相关程度，决定 $\boldsymbol{x}_j$ 对 $\boldsymbol{x}_i$ 的贡献。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-16)、(2-17)）

投影：Q、K、V 来自同一输入

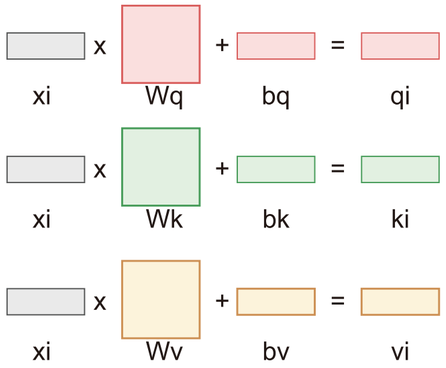


图 2.13(a)　投影变换

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_q + \boldsymbol{b}_q \tag{2-18}$$

$$\boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_k + \boldsymbol{b}_k \tag{2-19}$$

$$\boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_v + \boldsymbol{b}_v \tag{2-20}$$

$\boldsymbol{X} \in \mathbb{R}^{n\times d}$ 每行是一个词向量；$\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v \in \mathbb{R}^{d\times d}$ 为可学习的投影矩阵，$\boldsymbol{b}_q, \boldsymbol{b}_k, \boldsymbol{b}_v$ 加到每一行上。

- **自注意力**中，$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 都由同一个输入 $\boldsymbol{X}$ 投影得到。
- $\boldsymbol{Q}$、$\boldsymbol{K}$ 主要用于计算相关性，$\boldsymbol{V}$ 用于特征学习。
- 训练时投影矩阵可学习，从而改变词元之间的注意力分数。

图 《大语言模型：训练、精调与对齐》图 2.13(a)；式 (2-18)–(2-20)

注意力分数：查询与键的点积并缩放

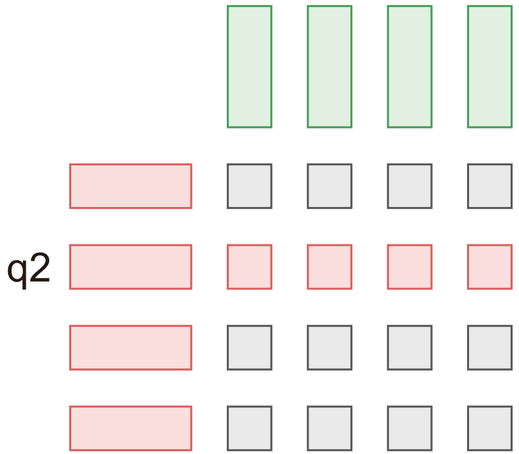


图 2.13(b)　注意力权重计算

$$\boldsymbol{S}_i = \boldsymbol{q}_i \boldsymbol{K}^T \tag{2-21}$$

第 i 个查询与所有键的分数行向量，$\boldsymbol{S}_i \in \mathbb{R}^{1\times n_k}$。

$$\boldsymbol{S} = \frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d}} \tag{2-22}$$

$\boldsymbol{S} \in \mathbb{R}^{n_q \times n_k}$；自注意力中 $n_q = n_k = n$；除以 $\sqrt{d}$ 控制分数的尺度。

- $\boldsymbol{S}$ 的第 i 行对应查询词元 i，第 j 列对应键词元 j。

图 《大语言模型：训练、精调与对齐》图 2.13(b)；式 (2-21)、(2-22)

为什么要除以 $\sqrt{d}$

$$\boldsymbol{q}\cdot\boldsymbol{k}=\sum_{m=1}^{d}q_m k_m$$

$$\mathbb{E}\left[q_m k_m\right]=\mathbb{E}\left[q_m\right]\mathbb{E}\left[k_m\right]=0$$

$$\operatorname{Var}\left(q_m k_m\right)=\mathbb{E}\left[\left(q_m k_m\right)^2\right]=\mathbb{E}\left[q_m^2\right]\mathbb{E}\left[k_m^2\right]=1$$

$$\operatorname{Var}\left(\boldsymbol{q}\cdot\boldsymbol{k}\right)=\sum_{m=1}^{d}\operatorname{Var}\left(q_m k_m\right)=d$$

$$\operatorname{Var}\left(\frac{\boldsymbol{q}\cdot\boldsymbol{k}}{\sqrt{d}}\right)=\frac{1}{d}\operatorname{Var}\left(\boldsymbol{q}\cdot\boldsymbol{k}\right)=1$$

假设 $\boldsymbol{q}$、$\boldsymbol{k}$ 的各分量相互独立，且均服从均值为 0、方差为 1 的标准正态分布。

不缩放

点积的标准差为 $\sqrt{d}$，维度越大，$|\boldsymbol{q}\cdot\boldsymbol{k}|$ 越大。

缩放后

除以 $\sqrt{d}$ 后方差回到 1，Softmax 的输入保持在梯度有效的范围内。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-22) 的缩放说明）

注意力权重与加权求和

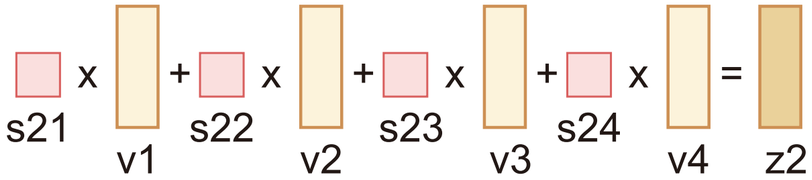


图 2.13(c)　特征加权组合

$$\boldsymbol{P}_i = \mathtt{Softmax}\left(\boldsymbol{S}_i\right) \tag{2-23}$$

$$\boldsymbol{z}_i = \sum_{j=1}^{n_k} P_{ij}\boldsymbol{v}_j = \boldsymbol{P}_i\boldsymbol{V} \tag{2-24}$$

$$\boldsymbol{Z} = \boldsymbol{P}\boldsymbol{V} \tag{2-25}$$

逐行 Softmax：$P_{ij} = \dfrac{\exp(S_{ij})}{\sum_k \exp(S_{ik})}$，每行之和为 1。

- 每个查询 i 都有一份独立的权重分布。
- $\boldsymbol{z}_i$ 是 $\boldsymbol{x}_i$ 融合上下文后的新表示；$\boldsymbol{Z} \in \mathbb{R}^{n_q \times d}$。

图 《大语言模型：训练、精调与对齐》图 2.13(c)；式 (2-23)–(2-25)

缩放点积注意力的完整流程



$$\boldsymbol{O} = \boldsymbol{Z}\boldsymbol{W}_o + \boldsymbol{b}_o \tag{2-26}$$

$\boldsymbol{W}_o \in \mathbb{R}^{d\times d}$、$\boldsymbol{b}_o \in \mathbb{R}^{d}$ 为可学习的线性投影，进一步增强表示能力；$\boldsymbol{O} \in \mathbb{R}^{n_q \times d}$。

合并上面各步，即 $\boldsymbol{Z} = \mathtt{Softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d}}\right)\boldsymbol{V}$。这就是本章 2.2 节给出的缩放点积注意力。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-18)–(2-26)）

数值示例：三个词元从分数矩阵到输出向量

给定序列（大、模、型），经输入层得到 $\boldsymbol{X} \in \mathbb{R}^{3\times d}$，再投影为 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$，逐元素算得分数 $s_{ij} = \boldsymbol{q}_i \boldsymbol{k}_j^T / \sqrt{d}$（式 (2-27)），即表 2.2；再按行做 Softmax、加权求和得到输出。

表 2.2　注意力计算实例（行对应查询，列对应键）

$\boldsymbol{q}_i \boldsymbol{k}_j^T / \sqrt{d}$	$\boldsymbol{k}_{大}$	$\boldsymbol{k}_{模}$	$\boldsymbol{k}_{型}$
$\boldsymbol{q}_{大}$	$s_{大,大}$	$s_{大,模}$	$s_{大,型}$
$\boldsymbol{q}_{模}$	$s_{模,大}$	$s_{模,模}$	$s_{模,型}$
$\boldsymbol{q}_{型}$	$s_{型,大}$	$s_{型,模}$	$s_{型,型}$

- 每个查询 $\boldsymbol{q}_i$ 与全部键 $\boldsymbol{k}_j$ 算一行分数，三个词元得到 3×3 的分数矩阵。
- 表中是 Softmax 之前的分数 s_{ij}。

$$\boldsymbol{P}_i = \texttt{Softmax}\left(\boldsymbol{S}_i\right) \tag{2-28}$$

$$\boldsymbol{P}_{大} = \texttt{Softmax}\left(\boldsymbol{S}_{大}\right) \tag{2-29}$$

$$\boldsymbol{z}_i = \boldsymbol{P}_i \boldsymbol{V} = \sum_j P_{ij} \boldsymbol{v}_j \tag{2-30}$$

$$\begin{aligned} \boldsymbol{z}_{大} = \boldsymbol{P}_{大} \boldsymbol{V} &= P_{大,大} \boldsymbol{v}_{大} + P_{大,模} \boldsymbol{v}_{模} \\ &\quad + P_{大,型} \boldsymbol{v}_{型} \end{aligned} \tag{2-31}$$

$$\boldsymbol{Z} = [\boldsymbol{z}_1; \boldsymbol{z}_2; \boldsymbol{z}_3] = [\boldsymbol{z}_{大}; \boldsymbol{z}_{模}; \boldsymbol{z}_{型}] \tag{2-32}$$

$\boldsymbol{Z} \in \mathbb{R}^{3\times d}$，与输入 $\boldsymbol{X}$ 的维度一致。$P_{大,大} + P_{大,模} + P_{大,型} = 1$，“大”的关注权重合计为 1。

多头注意力：并行的多个注意力头

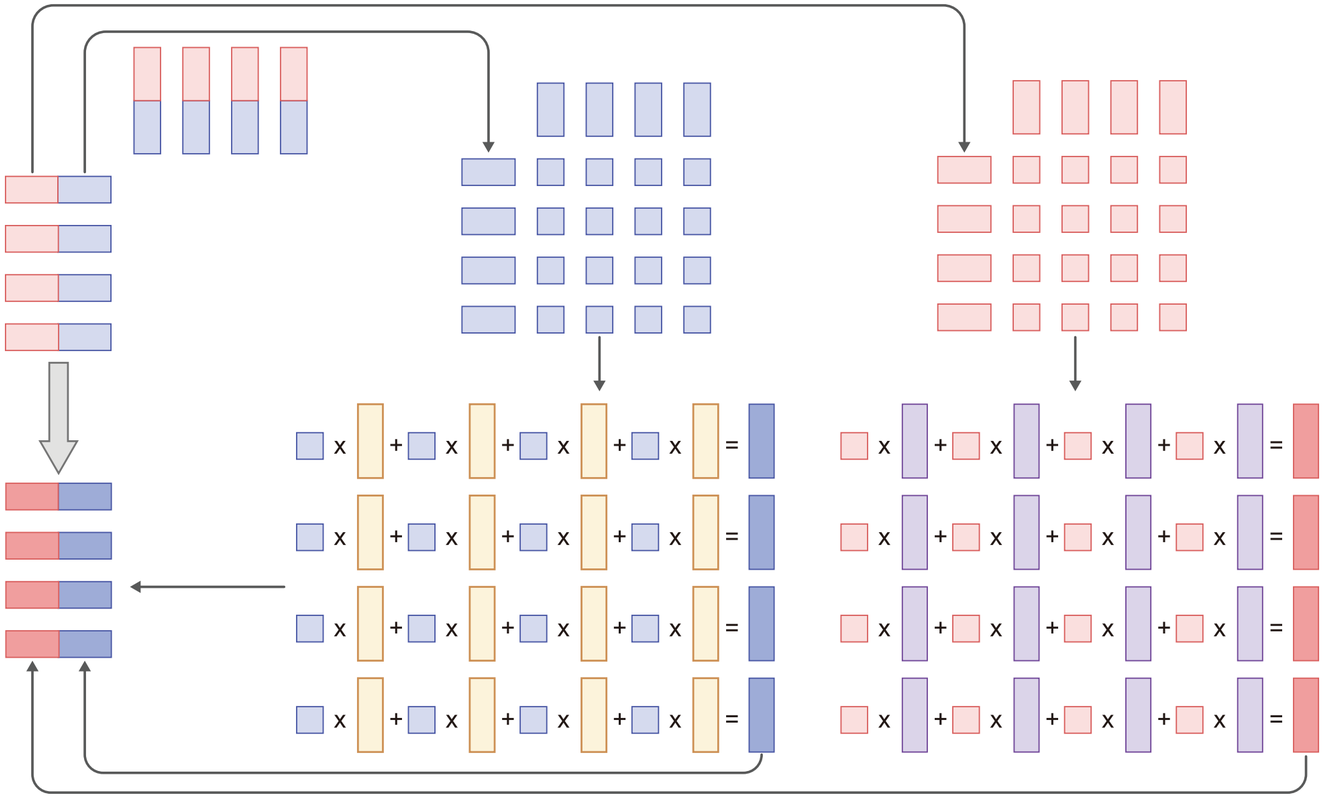


图 2.14　多头注意力机制（图示为两个注意力头）

- 单一注意力头的表示能力有限，上下文表示可能不够丰富。
- 沿特征维度切分后，各头在不同的表示子空间中学习不同的注意力模式。
- 各头彼此独立，可以并行计算，显著提高效率。

两个头各自计算一份 $\boldsymbol{S}$ 与 $\boldsymbol{Z}$，最后沿特征维拼接，再由 $\boldsymbol{W}_o$ 融合（式 (2-36)–(2-38)）。

沿特征维度切分 Q、K、V

$$\boldsymbol{Q} \to \left[\boldsymbol{Q}^{(1)}, \boldsymbol{Q}^{(2)}, \cdots, \boldsymbol{Q}^{(h)}\right], \quad \boldsymbol{Q}^{(i)} \in \mathbb{R}^{n \times d'} \tag{2-33}$$

$$\boldsymbol{K} \to \left[\boldsymbol{K}^{(1)}, \boldsymbol{K}^{(2)}, \cdots, \boldsymbol{K}^{(h)}\right], \quad \boldsymbol{K}^{(i)} \in \mathbb{R}^{n \times d'} \tag{2-34}$$

$$\boldsymbol{V} \to \left[\boldsymbol{V}^{(1)}, \boldsymbol{V}^{(2)}, \cdots, \boldsymbol{V}^{(h)}\right], \quad \boldsymbol{V}^{(i)} \in \mathbb{R}^{n \times d'} \tag{2-35}$$

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$ 沿特征维度均匀切分为 h 份，$d' = d/h$，要求 d 可被 h 整除。

- 切分前后词元数 n 不变，只有特征维度被分开：每个头处理 d' 维子空间。
- 例如 $d = 512$、$h = 8$ 时，每个头的维度为 $d' = 64$。

每个头独立计算，再拼接与投影

切分
$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \to h$ 份

→

并行 SDPA
每头 d' 维 (2-36)

→

拼接
沿特征维度 (2-37)

→

投影
$\boldsymbol{W}_o$ 融合 (2-38)

$$\boldsymbol{Z}^{(i)} = \texttt{Softmax}\left(\frac{\boldsymbol{Q}^{(i)}{\boldsymbol{K}^{(i)}}^T}{\sqrt{d'}}\right)\boldsymbol{V}^{(i)} \in \mathbb{R}^{n\times d'} \tag{2-36}$$

$$\boldsymbol{Z} \leftarrow \left[\boldsymbol{Z}^{(1)}, \boldsymbol{Z}^{(2)}, \cdots, \boldsymbol{Z}^{(h)}\right] \in \mathbb{R}^{n\times d} \tag{2-37}$$

$$\boldsymbol{O} = \boldsymbol{Z}\boldsymbol{W}_o + \boldsymbol{b}_o \in \mathbb{R}^{n\times d} \tag{2-38}$$

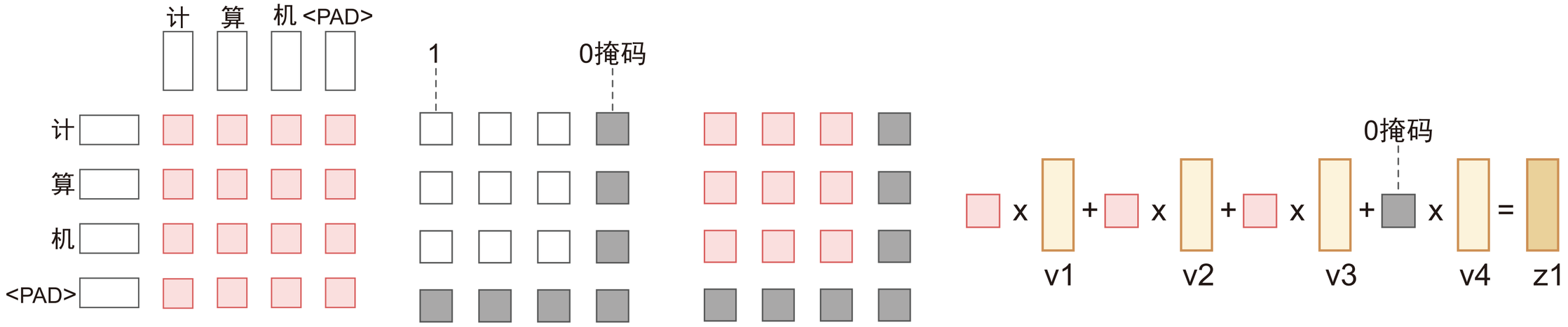
第 i 个头的缩放因子为 $\sqrt{d'}$（而非 $\sqrt{d}$）；拼接后的 $\boldsymbol{Z}$ 与输入维度相同。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-36)–(2-38)）

自注意力的掩码：屏蔽 <PAD>

$$M_{ij}^{\text{enc}} = \begin{cases} 1 & \text{if } (w_i \neq \text{<PAD>} \text{ and } w_j \neq \text{<PAD>}) \\ 0 & \text{otherwise} \end{cases} \tag{2-39}$$

$\boldsymbol{M}^{\text{enc}} \in \{0,1\}^{n_q \times n_k}$：$n_q$ 为查询词元数，n_k 为键词元数，自注意力中二者都等于序列长度 n。$M_{ij}^{\text{enc}} = 1$ 表示第 i 个查询可以读取第 j 个键，0 表示屏蔽。w_i 是第 i 个查询位置的词元，w_j 是第 j 个键位置的词元；任一侧为填充符 <PAD>，该位置即被屏蔽。



(a) 注意力分数　(b) 掩码矩阵　(c) 掩码分数　(d) 带掩码的注意力特征

图 2.15　掩码注意力机制：先由式 (2-22) 得到分数矩阵 (a)，再用掩码矩阵 (b) 把 <PAD> 对应的分数置为 $-\infty$ 得 (c)，Softmax 后这些位置的权重为 0，加权求和的输出见 (d)。

掩码的作用：被屏蔽的分数取 $-\infty$

$$S_{ij} = \begin{cases} S_{ij} & \text{if } M^{\text{enc}}_{ij} = 1 \\ -\infty & \text{if } M^{\text{enc}}_{ij} = 0 \end{cases} \tag{2-40}$$

先由式 (2-22) 得到原始分数 $\boldsymbol{S}$，再按掩码矩阵 $\boldsymbol{M}^{\text{enc}}$ 逐元素处理。

$$M^{\text{enc}}_{ij} = 0 \;\Rightarrow\; S_{ij} = -\infty \;\Rightarrow\; \exp(S_{ij}) = 0$$

$$P_{ij} = \frac{\exp(S_{ij})}{\sum_k \exp(S_{ik})} = \frac{0}{\sum_k \exp(S_{ik})} = 0$$

只要该行至少有一个未被屏蔽的分数，分母就不为零，被屏蔽位置的权重即为 0。

- 实现中通常不删除 <PAD> 对应的行与列，以保持权重与输出的维度一致，简化计算。
- 掩码只改变“哪些位置参与计算”，三类注意力的核心计算形式相同（见 2.3.5）。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-40)）

为什么用层归一化，而不是批归一化

归一化把每层的输入标准化为均值 0、方差 1 的分布，可以稳定训练，允许更大的学习率、加速收敛。

批归一化（BN）

在通道维度上归一化，统计同一位置、跨整个批量样本的均值与方差。

变长序列与 <PAD> 位置会让统计量不稳定、难以计算，因而在语言建模中效果不佳。

常用于计算机视觉的卷积网络。

层归一化（LN）

在每个词元的特征维度上归一化，每个词向量独立计算自身的均值与方差。

不受变长序列和批量大小的影响，适合语言模型。

特征通道上还有可学习的线性变换 γ、β，有助于加快收敛。

层归一化：每个词向量独立归一化

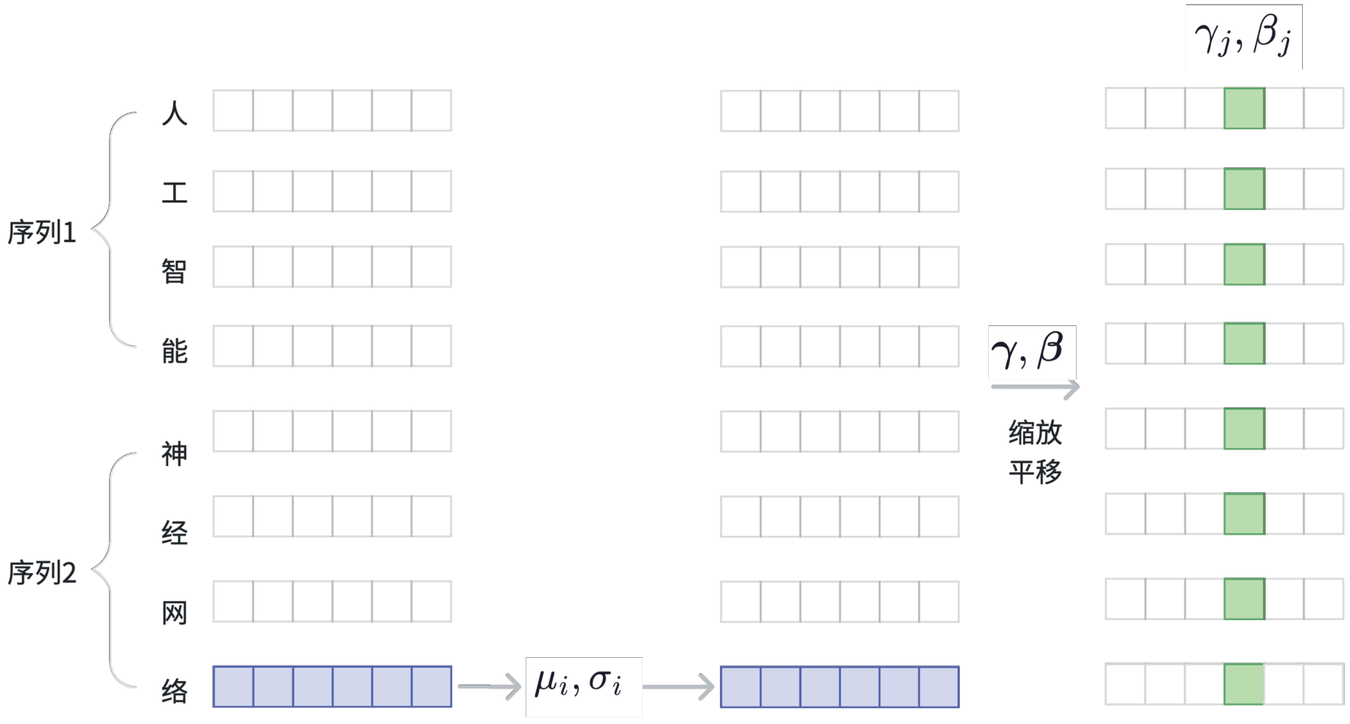


图 2.16　批量数据的层归一化

- 示例中两个序列分别为（人、工、智、能）与（神、经、网、络）。
- 每个词向量各自计算均值 μ_i 与方差 σ_i^2，不依赖其他序列。
- 归一化之后，在特征维度上做缩放 γ 与平移 β。

同一列的不同词元分属不同序列，互不参与对方的统计量——这是层归一化与批归一化的根本差别。

层归一化的计算：均值、方差与标准化

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij} \tag{2-41}$$

$$\sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d} (x_{ij} - \mu_i)^2 \tag{2-42}$$

$$\hat{\boldsymbol{x}}_i = \frac{\boldsymbol{x}_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \tag{2-43}$$

$\boldsymbol{x}_i$ 为第 i 个词元的 d 维特征，ϵ 为小的正实数，防止分母为零导致数值溢出。

第 1 步

求均值 μ_i（2-41）

→

第 2 步

求方差 σ_i^2（2-42）

→

第 3 步

标准化 $\hat{\boldsymbol{x}}_i$（2-43）

批量数据 $\boldsymbol{X} \in \mathbb{R}^{B\times L\times d}$ 中共有 $N = B \times L$ 个词元，上述运算对每个词元分别进行。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-41)–(2-43)）

可学习的缩放与平移，以及归一化的效果

$$\tilde{x}_{ij} = \gamma_j \hat{x}_{ij} + \beta_j \tag{2-44}$$

$\boldsymbol{\gamma} \in \mathbb{R}^d$ 初始化为全 1，$\boldsymbol{\beta} \in \mathbb{R}^d$ 初始化为全 0；对第 i 个词元的第 j 维做变换。$\boldsymbol{\gamma}$ 调整尺度，$\boldsymbol{\beta}$ 调整偏移。

$$\frac{1}{d}\sum_{j=1}^{d} \hat{x}_{ij} = \frac{1}{\sqrt{\sigma_i^2+\epsilon}} \cdot \frac{1}{d}\sum_{j=1}^{d}(x_{ij}-\mu_i) = \frac{\mu_i - \mu_i}{\sqrt{\sigma_i^2+\epsilon}} = 0$$

$$\frac{1}{d}\sum_{j=1}^{d} \hat{x}_{ij}^2 = \frac{1}{\sigma_i^2+\epsilon} \cdot \frac{1}{d}\sum_{j=1}^{d}(x_{ij}-\mu_i)^2 = \frac{\sigma_i^2}{\sigma_i^2+\epsilon} \approx 1$$

由式 (2-41)、(2-42) 代入式 (2-43) 得到：归一化后的每个词向量均值为 0；当 ϵ 远小于 σ_i^2 时，方差约为 1。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-44)；均值与方差的验证由式 (2-41)–(2-43) 推出）

前馈层：升维、激活，再降维

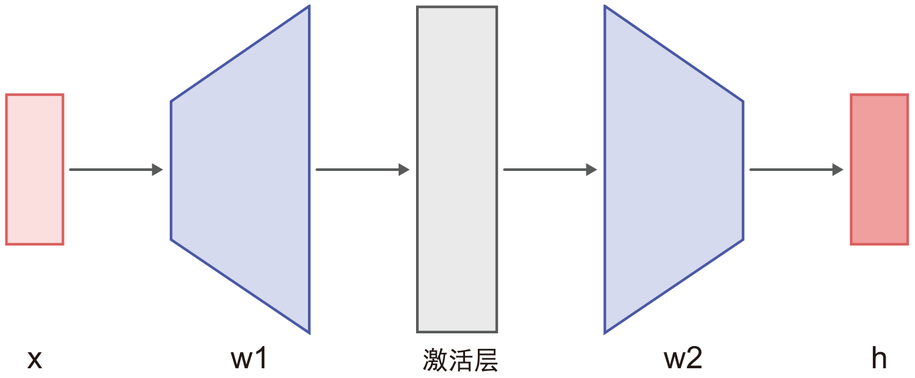


图 2.17　单个向量的前馈层计算示意图

升维

$\boldsymbol{W}_1 \in \mathbb{R}^{d\times 4d}$，$\boldsymbol{b}_1 \in \mathbb{R}^{4d}$，投影到更高维空间。

激活

$\mathrm{ReLU}(x) = \max(0, x)$，引入非线性。

降维

$\boldsymbol{W}_2 \in \mathbb{R}^{4d\times d}$，$\boldsymbol{b}_2 \in \mathbb{R}^{d}$，恢复为输入的维度。

前馈层的计算：逐位置独立进行

$$\boldsymbol{X}_{\text{up}} = \boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{b}_1 \tag{2-45}$$

$$\boldsymbol{X}_{\text{act}} = \texttt{ReLU}\left(\boldsymbol{X}_{\text{up}}\right) \tag{2-46}$$

$$\boldsymbol{X}_{\text{down}} = \boldsymbol{X}_{\text{act}}\boldsymbol{W}_2 + \boldsymbol{b}_2 \tag{2-47}$$

$$\boldsymbol{h}_i = \left(\texttt{ReLU}\left(\boldsymbol{x}_i\boldsymbol{W}_1 + \boldsymbol{b}_1\right)\right)\boldsymbol{W}_2 + \boldsymbol{b}_2 \tag{2-48}$$

式 (2-45)–(2-47) 的形状依次为 $n \times 4d$、$n \times 4d$、$n \times d$；式 (2-48) 对单个词向量 $\boldsymbol{x}_i \in \mathbb{R}^{1\times d}$ 独立计算，无需与其他词元交互。

- 升维提供更大的容量，学习更复杂的非线性特征；降维控制复杂度，并把维度恢复为 d，便于堆叠。

依据 《大语言模型：训练、精调与对齐》第 2.3.4 节（式 (2-45)–(2-48)）

编码器的整体：多层编码块的上下文融合

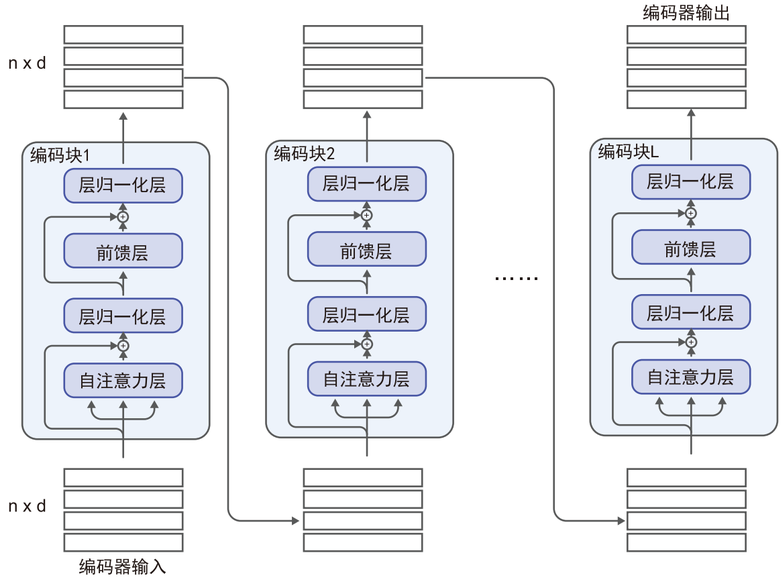


图 2.18　多层编码块计算过程

- 第一个自注意力层后，每个词元已按相关性加权融合了其他词的信息。
- 后续编码块反复做这种加权融合，捕捉更复杂的依赖关系。
- $\boldsymbol{X}_{\text{enc}} \in \mathbb{R}^{n \times d}$ 中每个词元都带有整个输入序列的语义。
- 与 RNN 编码器相比，它为每个词元保留一个编码特征，并行计算；RNN 通过递归串行计算，通常只得到一个（或一组）隐向量。

解码器：自回归生成的 L 个解码块

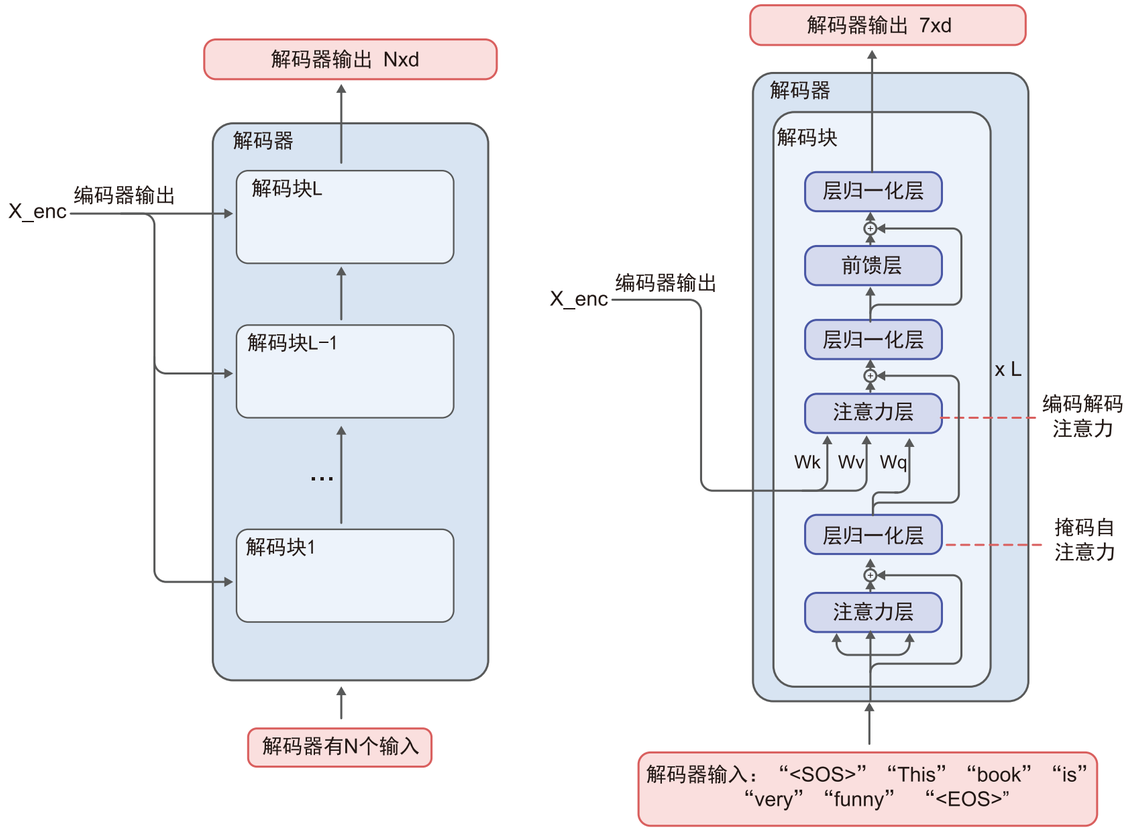


图 2.19　解码器结构，解码器的输入为目标翻译文本的词元序列

$$\boldsymbol{Y}^{(l)} = \texttt{Decoder_Block}^{(l)}\left(\boldsymbol{X}_{\text{enc}}, \boldsymbol{Y}^{(l-1)}\right) \tag{2-49}$$

第 l 个解码块同时读取编码器输出 $\boldsymbol{X}_{\text{enc}}$ 与自身输入 $\boldsymbol{Y}^{(l-1)}$；$\boldsymbol{Y}^{(0)} = \boldsymbol{Y} \in \mathbb{R}^{m\times d}$，$m$ 为目标序列长度。

- 任务：根据编码器对源语言的理解，以及目标语言前 t 个词元，预测第 $t+1$ 个词。
- 训练时，整个目标序列可以并行输入，每个位置各自预测下一个词。

解码块：三类层与残差连接

解码块在编码块的基础上多了一次“编码-解码注意力”，使解码器在生成每个词时读取源语言的信息。

子层 1

掩码自注意力层

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 均来自 $\boldsymbol{Y}^{(l-1)}$（同一输入）。因果掩码 $\boldsymbol{M}^{\text{dec}}$ 使位置 t 只能看到 $\leqslant t$ 的词元，防止信息泄露。

子层 2

编码-解码注意力层

$\boldsymbol{Q}$ 来自解码块特征 $\boldsymbol{Y}_{\text{dec}}$；$\boldsymbol{K}, \boldsymbol{V}$ 来自编码器输出 $\boldsymbol{X}_{\text{enc}}$，用于学习源语言与目标语言之间的对齐。

其他

LN、前馈层与残差

层归一化层（`Layer_Norm_1/2/3`）、前馈层与残差连接，与编码块相同。

依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（解码块的三类层）

解码块（一）：掩码自注意力子层

$$\boldsymbol{Y}_{\text{attn}} = \texttt{Multi_Head_Attention}\left(\boldsymbol{Y}^{(l-1)}, \boldsymbol{M}^{\text{dec}}\right) \tag{2-50}$$

$$\boldsymbol{Y} = \boldsymbol{Y}_{\text{attn}} + \boldsymbol{Y}^{(l-1)} \tag{2-51}$$

$$\boldsymbol{Y}_{\text{ln1}} = \texttt{Layer_Norm_1}\left(\boldsymbol{Y}\right) \tag{2-52}$$

结构与式 (2-10)–(2-12) 的编码块注意力子层相同：先做残差相加，再归一化。区别在于掩码矩阵为因果掩码 $\boldsymbol{M}^{\text{dec}}$。

- $\boldsymbol{Y}^{(l-1)}$ 同时作为 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 的来源，因此这是自注意力。
- $\boldsymbol{M}^{\text{dec}}$ 决定每个位置能够看到哪些已生成的词元（见 2.3.5 掩码相关页）。

依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（式 (2-50)–(2-52)）

解码块（二）：编码-解码注意力与前馈子层

$$\boldsymbol{Y}_{\text{attn}} = \texttt{Encoder_Decoder_Attention}\left(\boldsymbol{Y}_{\text{ln1}}, \boldsymbol{X}_{\text{enc}}, \boldsymbol{M}^{\text{enc-dec}}\right) \tag{2-53}$$

$$\boldsymbol{Y}' = \boldsymbol{Y}_{\text{attn}} + \boldsymbol{Y}_{\text{ln1}} \tag{2-54}$$

$$\boldsymbol{Y}_{\text{ln2}} = \texttt{Layer_Norm_2}\left(\boldsymbol{Y}'\right) \tag{2-55}$$

$$\boldsymbol{Y}_{\text{ffn}} = \texttt{Feed_Forward}\left(\boldsymbol{Y}_{\text{ln2}}\right) \tag{2-56}$$

$$\boldsymbol{Y}'' = \boldsymbol{Y}_{\text{ffn}} + \boldsymbol{Y}_{\text{ln2}} \tag{2-57}$$

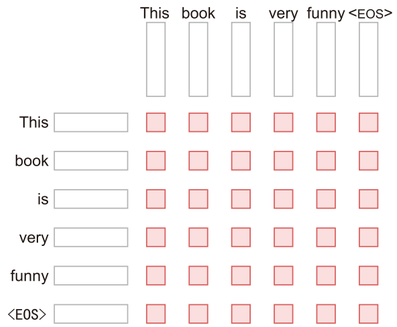
$$\boldsymbol{Y}^{(l)} = \texttt{Layer_Norm_3}\left(\boldsymbol{Y}''\right) \tag{2-58}$$

三个子层都是“先残差相加、再归一化”：$\boldsymbol{Y}'$、$\boldsymbol{Y}''$ 为残差相加后的中间结果。编码-解码注意力的 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 来源见下一节。

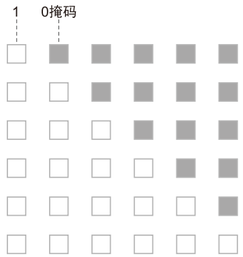
依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（式 (2-53)–(2-58)）

掩码自注意力：因果掩码 M^{dec}

图 2.20　掩码自注意力层的掩码矩阵示意图



(a) 注意力分数



(b) 掩码矩阵：主对角线及以下为 1，以上为 0

$$M_{ij}^{\text{dec}} = \begin{cases} 1 & \text{if } (w_i \neq \text{<PAD>} \text{ and } w_j \neq \text{<PAD>} \\ & \quad \text{and } j \leqslant i) \\ 0 & \text{otherwise} \end{cases} \tag{2-59}$$

只允许位置 i 关注位置 1 到 i 的词元（当前及之前），屏蔽未来位置 $j > i$。

- 深灰色区域对应 $M_{ij}^{\text{dec}} = 0$，其分数随后被置为 $-\infty$（见下页）。
- 矩阵是下三角结构：位置越靠后，可读取的前文越完整。
- 同时出现的 <PAD> 条件用于批量训练时的填充位置。

图 《大语言模型：训练、精调与对齐》图 2.20；式 (2-59)

因果掩码如何作用于注意力分数

$$S_{ij} = \begin{cases} S_{ij} & \text{if } M_{ij}^{\text{dec}} = 1 \\ -\infty & \text{if } M_{ij}^{\text{dec}} = 0 \end{cases} \tag{2-60}$$

$$\boldsymbol{Y}' = \texttt{Multi_Head_Attention}\left(\boldsymbol{Y}, \boldsymbol{M}^{\text{dec}}\right) \tag{2-61}$$

$\boldsymbol{Y}'$ 为当前层的解码特征；式 (2-60) 中被屏蔽的 S_{ij} 取 $-\infty$。

$$\begin{aligned} \boldsymbol{S}_i &= (S_{i1}, \cdots, S_{ii}, -\infty, \cdots, -\infty) \\ \boldsymbol{P}_i &= \texttt{Softmax}\left(\boldsymbol{S}_i\right) = (P_{i1}, \cdots, P_{ii}, 0, \cdots, 0) \\ \boldsymbol{z}_i &= \boldsymbol{P}_i \boldsymbol{V} = \sum_{j \leqslant i} P_{ij} \boldsymbol{v}_j \end{aligned}$$

以第 i 行为例：$j > i$ 的位置权重为 0，$\boldsymbol{z}_i$ 只由位置 1 到 i 的值向量组成。

编码-解码注意力：Q 来自解码器，K、V 来自编码器

$$\boldsymbol{Q}_{\text{dec}} = \boldsymbol{Y}_{\text{dec}}\boldsymbol{W}_q + \boldsymbol{b}_q \tag{2-62}$$

$$\boldsymbol{K}_{\text{enc}} = \boldsymbol{X}_{\text{enc}}\boldsymbol{W}_k + \boldsymbol{b}_k \tag{2-63}$$

$$\boldsymbol{V}_{\text{enc}} = \boldsymbol{X}_{\text{enc}}\boldsymbol{W}_v + \boldsymbol{b}_v \tag{2-64}$$

$$\boldsymbol{S} = \frac{\boldsymbol{Q}_{\text{dec}}\boldsymbol{K}_{\text{enc}}^{T}}{\sqrt{d}} \tag{2-65}$$

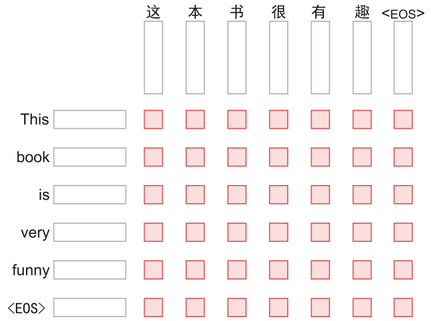
$\boldsymbol{X}_{\text{enc}} \in \mathbb{R}^{n\times d}$，$\boldsymbol{Y}_{\text{dec}} \in \mathbb{R}^{m\times d}$；$\boldsymbol{Q}_{\text{dec}} \in \mathbb{R}^{m\times d}$，$\boldsymbol{K}_{\text{enc}}, \boldsymbol{V}_{\text{enc}} \in \mathbb{R}^{n\times d}$，$\boldsymbol{S} \in \mathbb{R}^{m\times n}$。式 (2-65) 为非多头形式，实际以多头实现。

- 每个目标位置（查询）与全部源语言位置（键）计算分数，因此 $\boldsymbol{S}$ 是 $m \times n$ 的矩形矩阵。
- n 为源语言长度，m 为目标语言长度，二者不必相等。

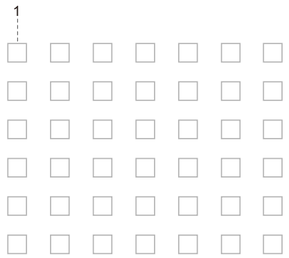
依据 《大语言模型：训练、精调与对齐》第 2.3.5 节（式 (2-62)–(2-65)）

编码-解码注意力的填充掩码

图 2.21　编码器-解码器注意力掩码矩阵（示例：源语言“这本书很有趣<EOS>”，目标语言 This book is very funny <EOS>）



(a) 注意力分数



(b) 掩码矩阵

$$M_{ij}^{\text{enc-dec}} = \begin{cases} 1 & \text{if } (w_i \neq <\text{PAD}> \\ & \quad \text{and } w_j \neq <\text{PAD}>) \\ 0 & \text{otherwise} \end{cases} \tag{2-66}$$

$M^{\text{enc-dec}} \in \{0,1\}^{m \times n}$；$w_i$ 来自目标语言第 i 个位置，w_j 来自源语言第 j 个位置。

- 与因果掩码不同：掩码里没有 $j \leqslant i$ 的条件，任何目标位置都能看到全部源语言位置。
- 矩阵为 $m \times n$，行数等于目标序列长度，列数等于源序列长度。
- 被屏蔽的列是源序列末尾的 <PAD> 填充位，避免把填充信息读入解码。

三种注意力：计算形式相同，来源与掩码不同

编码器自注意力　式 (2-10)、(2-39)

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 同源

三者都取自 $\boldsymbol{X}^{(l-1)}$，在源序列内部读取。

掩码 $\boldsymbol{M}^{\text{enc}}$ 屏蔽 <PAD>。

解码器掩码自注意力　式 (2-50)、(2-59)

$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 同源

三者都取自 $\boldsymbol{Y}^{(l-1)}$，在目标序列内部读取。

掩码 $\boldsymbol{M}^{\text{dec}}$ 只看当前及之前。

编码-解码注意力　式 (2-53)、(2-66)

$\boldsymbol{Q}$ 与 $\boldsymbol{K}, \boldsymbol{V}$ 异源

$\boldsymbol{Q}$ 取自 $\boldsymbol{Y}_{\text{dec}}$，$\boldsymbol{K}, \boldsymbol{V}$ 取自 $\boldsymbol{X}_{\text{enc}}$。

掩码 $\boldsymbol{M}^{\text{enc-dec}}$ 屏蔽两端 <PAD>。

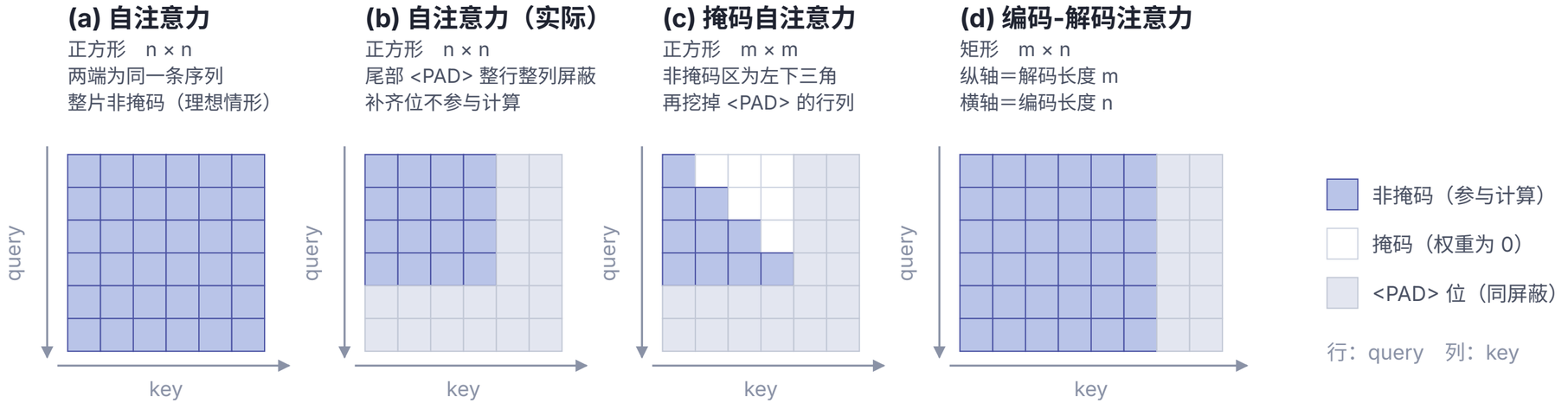
$$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \longrightarrow \boldsymbol{S} = \frac{\boldsymbol{Q}\boldsymbol{K}^{T}}{\sqrt{d}} \xrightarrow{\text{掩码}} \boldsymbol{P} = \texttt{Softmax}(\boldsymbol{S}) \longrightarrow \boldsymbol{Z} = \boldsymbol{P}\boldsymbol{V}$$

主线的四步对三者完全相同；只有掩码这一步，三张掩码矩阵的构造不同（式 (2-39)、(2-59)、(2-66)）。

注　三者的核心计算都是 $\texttt{Softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{T}}{\sqrt{d}}\right)\boldsymbol{V}$；差别只在于 $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$ 从哪里来，以及掩码矩阵如何构造。

三种注意力的掩码形态

把注意力分数矩阵画成网格：**行是 query、列是 key**。矩阵的**形状**由两端长度决定，**哪一片被遮住**由掩码决定；被屏蔽的格子在 Softmax 后权重为 0。实际批次会把序列补齐到同一长度，(a) 那样的整片可见几乎不存在——(b)(c) 才是实际的样子：尾部 <PAD> 整行整列屏蔽，否则补齐位会污染注意力。



形状

自注意力两端都是序列长度，因此是方阵；(d) 两端分别是目标与源，因此是矩形，行数与列数不必相等。

方向

行永远是 query、列永远是 key。屏蔽**整行**则该 query 读不到任何东西（<PAD> 行正是如此）；屏蔽**整列**则该 key 对所有 query 都不可见。

共同点

三者都从左上角起排：因果掩码靠“只留下三角”排除未来位置，(d) 不设上下三角；<PAD> 则在右、下两侧各留一圈浅灰带。

依据第 2.3.4、2.3.5 节；掩码定义见式 (2-39)、(2-59)、(2-66)

输出层：线性投影与 Softmax

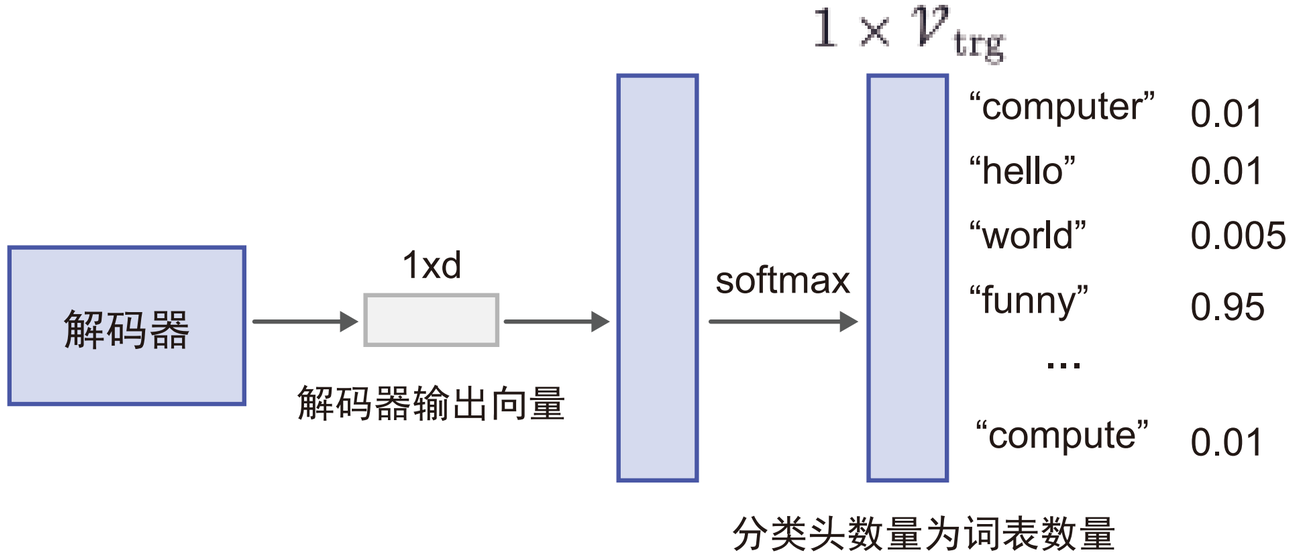


图 2.22(a)　下一个词元预测

$$p_\theta(\cdot \mid \boldsymbol{x}, \boldsymbol{y}_{\leqslant t}) = \texttt{Softmax}\left(\boldsymbol{Y}_t' \boldsymbol{W}_{\text{out}} + \boldsymbol{b}_{\text{out}}\right)$$

式 (2-67)：单个位置 t 的分布 $\in \mathbb{R}^{1\times|\mathcal{V}_{\text{trg}}|}$；整个序列为 $m \times |\mathcal{V}_{\text{trg}}|$。

- 语言分类头：$\boldsymbol{W}_{\text{out}} \in \mathbb{R}^{d\times|\mathcal{V}_{\text{trg}}|}$，$\boldsymbol{b}_{\text{out}} \in \mathbb{R}^{|\mathcal{V}_{\text{trg}}|}$。
- $|\mathcal{V}_{\text{trg}}|$ 为目标语言分词器的词表大小。
- Softmax 把投影结果转换为下一个词元的概率分布。

图 《大语言模型：训练、精调与对齐》图 2.22(a)；式 (2-67)

并行预测：一次计算所有位置的分布

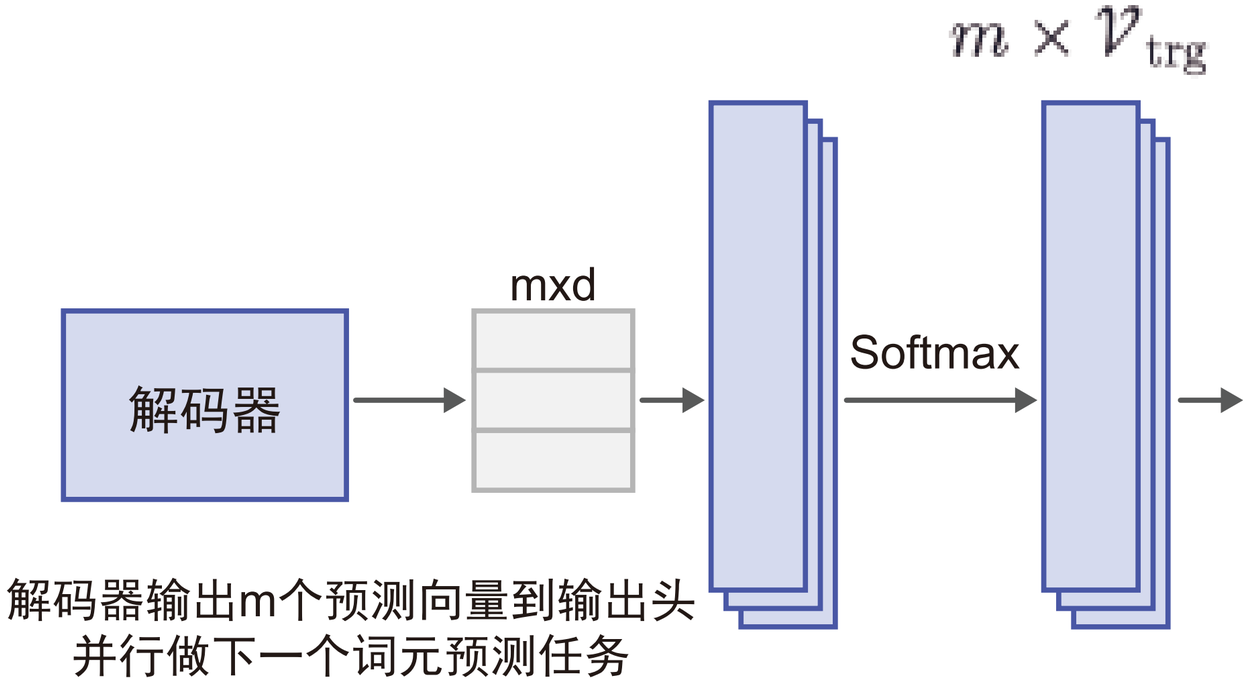


图 2.22(b)　并行预测

$$\boldsymbol{Y}' \in \mathbb{R}^{3\times d}$$

解码器输出（3 个位置）

$$\boldsymbol{Y}'\boldsymbol{W}_{\text{out}} + \boldsymbol{b}_{\text{out}} \in \mathbb{R}^{3\times|\mathcal{V}_{\text{trg}}|}$$

逐行 Softmax

每行为一个位置的概率分布

- 训练时给定完整前缀 $\boldsymbol{y}_{\leqslant m}$，式 (2-67) 对每个位置 t 并行计算 y_{t+1} 的分布。
- 可以把 Transformer 看作一个类别数极大的分类预测机器。

图 《大语言模型：训练、精调与对齐》图 2.22(b)；式 (2-67)

2.4

Transformer 训练和推理

本节内容

2.4.1 Transformer 训练

2.4.2 Transformer 推理

训练时一次前向计算并行预测所有位置；推理时则从 <SOS> 出发，逐词生成直至 <EOS>。

训练并行，推理串行

机器翻译中，模型的输入是源语言序列 $\boldsymbol{x}$ 与已生成的目标前缀 $\boldsymbol{y}_{<t}$，目标是预测下一个目标词元 y_t。训练与推理对目标序列的使用方式不同。

训练：并行

目标序列 $\boldsymbol{y}$ 整体作为解码器输入（教师强制），解码器一次前向计算所有位置的下一词元分布。

每个位置与其标签比较并计算交叉熵，反向传播后用 ADAM 优化器更新参数。

推理：串行

目标序列未知，只能从起始符 <SOS> 开始，逐词生成。

每步把预测词元追加到解码器输入，直至生成 <EOS> 或达到最大序列长度。

> **要点** 训练时的并行性来自注意力机制与掩码（2.4.1）；推理时的串行性来自“下一个词元依赖上一个词元”（2.4.2）。

一次前向计算预测所有位置

Transformer 通过单次前向计算即可同时预测目标序列中所有位置的下一词元分布，再用逐位置的交叉熵训练。

目标序列
含 <SOS>、<EOS>

→

单次前向
掩码自注意力与交叉注意力

→

逐位置分布
式 (2-67)

→

交叉熵损失
式 (2-5)

→

ADAM 更新
反向传播后更新参数

$$p_\theta(\cdot \mid \boldsymbol{x}, \boldsymbol{y}_{\leqslant t}) = \texttt{Softmax}(\boldsymbol{Y}'_t \boldsymbol{W}_{\text{out}} + \boldsymbol{b}_{\text{out}})$$

式 (2-67)：解码器第 t 个位置的输出 $\boldsymbol{Y}'_t$ 经输出层得到下一词元的分布（2.3 节）。

$$\mathcal{L}^{(k)}(\theta) = -\frac{1}{T_k} \sum_{t=1}^{T_k} \left[\log p_\theta(y_{t+1}^{(k)} \mid \boldsymbol{x}^{(k)}, \boldsymbol{y}_{\leqslant t}^{(k)}) \right]$$

式 (2-5)：样本 k 的负对数似然损失；$\frac{1}{T_k}$ 对各个位置取平均。

依据 《大语言模型：训练、精调与对齐》式 (2-5)、(2-67)

并行训练示意图

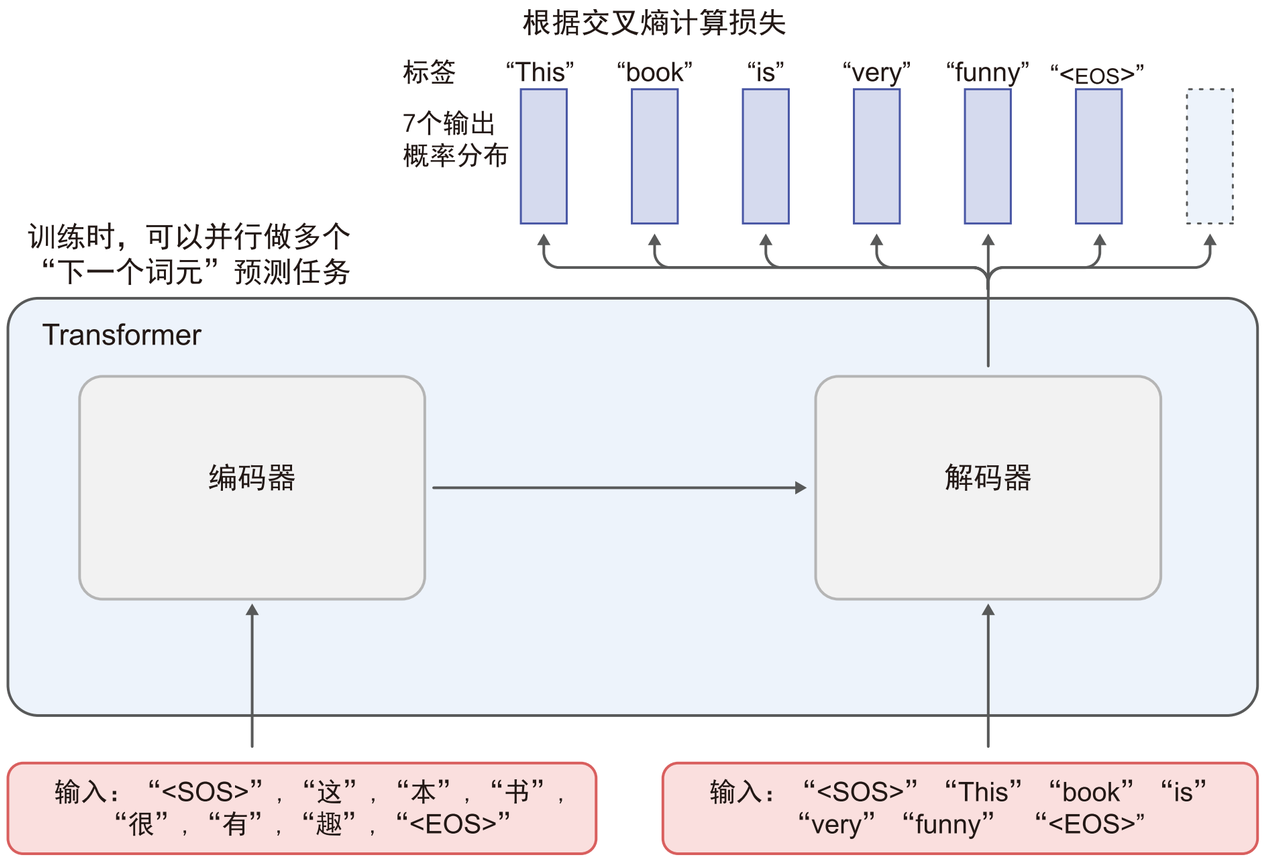


图 2.23　Transformer 并行训练示意图

- 编码器读入源语言 8 个词元（含 <SOS>、<EOS>），解码器读入目标序列 7 个词元。
- 解码器的 7 个输出位置各有一个概率分布：图中 6 个实线框标注了 This 至 <EOS>，右侧虚线框无标签。
- 末位 <EOS> 之后没有下一词元标签，不参与损失（标签为 <PAD> 的位置同样不参与），因此共 6 个预测任务。

图源 《大语言模型：训练、精调与对齐》图 2.23

输入与标签：标签左移一位

把目标序列（含 <SOS> 与 <EOS>）整体送入解码器；每个位置的标签是输入左移一位后的词元，即该位置的“下一个词元”。

位置 t	1	2	3	4	5	6	7
解码器输入	<SOS>	This	book	is	very	funny	<EOS>
标签（下一词元）	This	book	is	very	funny	<EOS>	无
参与损失	是	是	是	是	是	是	否

输入

目标序列加上 <SOS> 与 <EOS> 后共 7 个词元，整体作为解码器输入。

标签

位置 t 的标签是第 $t+1$ 个词元，即输入左移一位。

损失

末位无标签；下一词元标签为 <PAD> 的位置同样不参与计算。

依据 《大语言模型：训练、精调与对齐》第 2.4.1 节（并行训练示例）

掩码：训练时屏蔽后续词元

$$S_{ij} = \begin{cases} S_{ij} & \text{if } M^{\text{dec}}_{ij} = 1 \\ -\infty & \text{if } M^{\text{dec}}_{ij} = 0 \end{cases}$$

式 (2-60)：解码器因果掩码 M^{dec}_{ij} 在主对角线及以下为 1（保留），以上为 0（屏蔽）。

表 2.3　掩码注意力计算实例

$\boldsymbol{q}_i \boldsymbol{k}_j^T / \sqrt{d}$	$\boldsymbol{k}_1$	$\boldsymbol{k}_2$	$\boldsymbol{k}_3$
$\boldsymbol{q}_1$	s_{11}	$-\infty$	$-\infty$
$\boldsymbol{q}_2$	s_{21}	s_{22}	$-\infty$
$\boldsymbol{q}_3$	s_{31}	s_{32}	s_{33}

- $i < j$ 的分数 s_{ij} 被因果掩码置为 $-\infty$，对应 $M^{\text{dec}}_{ij} = 0$。
- Softmax 中 $e^{-\infty} = 0$，这些位置的注意力权重为 0。
- 查询 $\boldsymbol{q}_i$ 因此看不到 $\boldsymbol{y}_{>i}$，训练仍能并行计算，又不会泄露未来词元。

依据 《大语言模型：训练、精调与对齐》第 2.4.1 节；掩码定义见第 2.3.5 节

推理：从 <SOS> 开始逐词生成

训练完成后用于翻译时，目标序列未知。模型以自回归方式串行生成：每一步基于源语言 $\boldsymbol{x}$ 与已生成的前缀预测下一个词元，追加后再继续。

编码器

源语言序列只需单次前向计算，得到编码特征；之后每一步解码都复用这份特征。

解码器

初始输入只有起始符 <SOS>；输入为空则无法进行前向计算。

$$y_{\text{“This”}} = \arg\max_{w \in \mathcal{V}} p_\theta(w \mid \boldsymbol{x}, <\text{SOS}>)$$

式 (2-68)：以条件概率 $p_\theta(\cdot \mid \boldsymbol{x}, <\text{SOS}>)$ 最大的词元作为首词元（arg max，贪心选择）。

预测结果 $y_{\text{“This”}}$ 被追加到解码器输入，形成新输入 (<SOS>, This)，然后预测下一个词元。

串行推理示意图

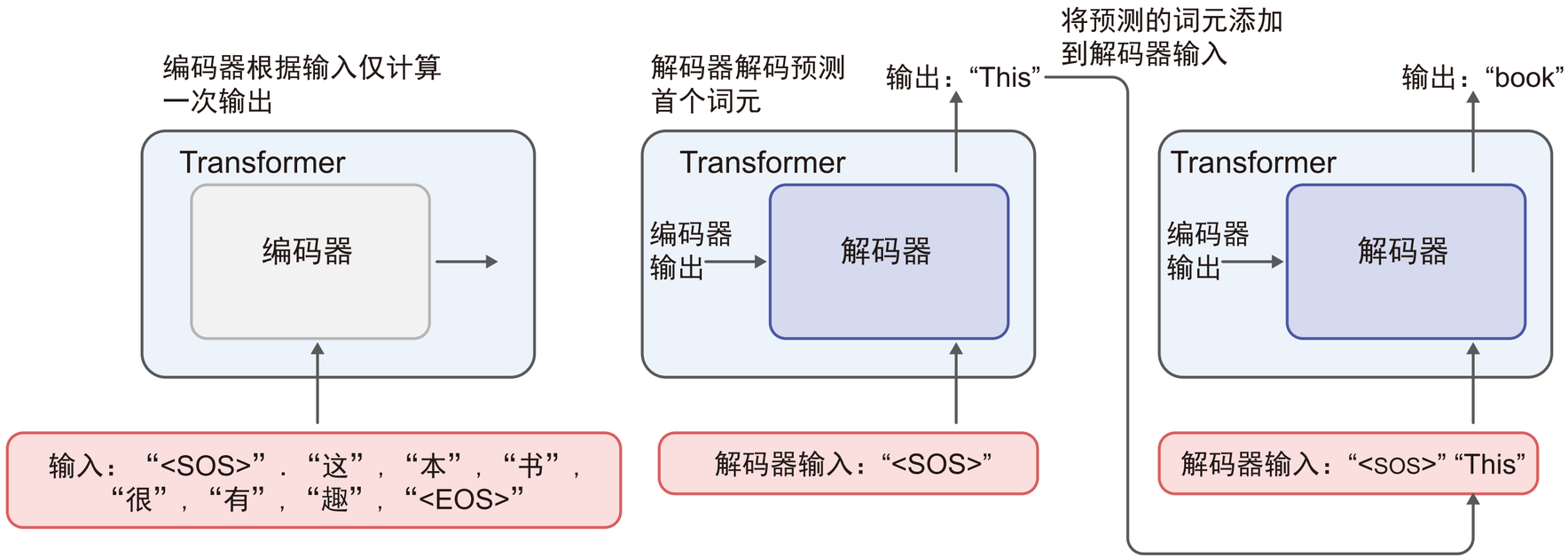


图 2.24　Transformer 串行推理

第 1 步

编码器对源语言只计算一次，输出全程不变。

第 2 步

解码器输入 <SOS>，预测首词元 This。

第 3 步

追加 This 得到 (<SOS>, This)，再预测 book。

图源 《大语言模型：训练、精调与对齐》图 2.24

逐词生成直到 <EOS>

输入 <SOS>
解码器首次输入

→

输出 This
选择概率最高的词元

→

输入 <SOS> This
追加预测词元

→

输出 book
继续预测下一词元

→

..... <EOS>
生成终止符即停止

- 基于更新后的输入序列预测下一词元，并将新词元追加至解码器输入序列。
- 迭代直至生成终止符 <EOS>，或达到最大序列长度。

代价 每次预测都要重新计算已生成词元的中间表示，计算冗余明显；4.3 节的 KV-Cache 通过缓存历史状态优化这一问题。

2.5

关键代码实践

用 PyTorch 逐段实现缩放点积注意力与多头注意力，并对照公式解释每一行代码。

本节内容

2.5.1 缩放点积注意力

2.5.2 多头注意力机制

代码 1　函数结构：一个函数完成一次注意力计算

代码 1 只有一个函数 `ScaledDotProductAttention`，加一段测试数据。计算部分共 8 行，依次是取形状、转置、点积、掩码、Softmax、加权求和、返回。

取形状
第 7 行
B, H, L, D

→

转置 K
第 8 行
`transpose(2,3)`

→

点积缩放
第 9 行
`Q @ Kt / √D`

→

掩码填充
第 10–11 行
`masked_fill`

→

Softmax
第 12 行
沿最后一维

→

加权求和
第 13–14 行
`P @ V`

函数签名

`(Q, K, V, mask)`：四个参数都是张量，`mask` 允许为 `None`。

张量形状

`(B, H, L, D)`：批、头、序列长度、每头维度。函数按张量运算，不关心具体的 B、H。

代码 1　函数与实现细节

```
 1  import torch
 2  import torch.nn as nn
 3  import math
 4  torch.manual_seed(42)
 5
 6  def ScaledDotProductAttention(Q, K, V, mask):
 7      B, H, L, D = K.size()
 8      Kt = K.transpose(2, 3)
 9      S = Q @ Kt / math.sqrt(D)
10      if mask is not None:
11          S = S.masked_fill(mask, -torch.tensor(float('inf')))
12      P = nn.functional.softmax(S, dim=-1)
13      Z = P @ V
14      return Z
```

- `K.size()` 解包出批、头、序列长度、每头维度四个量，D 只用于第 9 行的缩放。
- `transpose(2, 3)` 把键矩阵的最后两维对调，使 `Q @ Kt` 对每个批、每个头做一次矩阵乘法。
- `masked_fill` 把布尔掩码为 True 的位置写成 $-\infty$，Softmax 后这些位置的权重即为 0。
- `dim=-1` 指定沿最后一维归一化，即每一行分数各自成为一组权重。
- 函数内不做输出投影，返回的 `Z` 是加权求和的结果；投影放在多头注意力的类里。

依据 《大语言模型：训练、精调与对齐》代码 1（2.5.1 节）

代码 1　测试数据与掩码

```
15
16  # 模拟数据
17  bs = 4  # 批大小
18  heads = 8  # 注意力头数
19  seq_len = 16  # 序列长度
20  dim = 64  # 词嵌入维度
21  Q = torch.randn(bs, heads, seq_len, dim)
22  K = torch.randn(bs, heads, seq_len, dim)
23  V = torch.randn(bs, heads, seq_len, dim)
24  M = torch.zeros(1, 1, seq_len, seq_len, dtype=torch.bool)
25  out = ScaledDotProductAttention(Q, K, V, M)
```

- Q、K、V 形状为 (4, 8, 16, 64)：4 个样本、8 个头、长度 16、每头维度 64。
- 掩码形状 (1, 1, 16, 16)，通过广播作用于所有样本与所有头。
- 布尔掩码中 True 表示屏蔽，对应式 (2-40) 中 $M = 0$ 的位置；全为 False 时不屏蔽任何位置。
- 若掩码全为 True，每行分数都是 $-\infty$，Softmax 输出为 NaN。

依据 《大语言模型：训练、精调与对齐》代码 1（2.5.1 节）；掩码定义见式 (2-40)

多头注意力：切分、并行、拼接、投影

步骤 1 · 切分　式 (2-33)–(2-35)

$$\boldsymbol{Q} \to [\boldsymbol{Q}^{(1)}, \cdots, \boldsymbol{Q}^{(h)}], \quad \boldsymbol{Q}^{(i)} \in \mathbb{R}^{n \times d'}$$

$\boldsymbol{K}$、$\boldsymbol{V}$ 同理；$d' = d/h$，要求 d 能被 h 整除。

步骤 2 · 并行计算　式 (2-36)

$$\boldsymbol{Z}^{(i)} = \texttt{Softmax}\left(\frac{\boldsymbol{Q}^{(i)}\boldsymbol{K}^{(i)^T}}{\sqrt{d'}}\right)\boldsymbol{V}^{(i)}$$

每个头独立做缩放点积注意力，各头可同时计算。

步骤 3 · 拼接　式 (2-37)

$$\boldsymbol{Z} \leftarrow [\boldsymbol{Z}^{(1)}, \cdots, \boldsymbol{Z}^{(h)}] \in \mathbb{R}^{n \times d}$$

沿特征维拼接各头输出，恢复为 $n \times d$。

步骤 4 · 投影　式 (2-38)

$$\boldsymbol{O} = \boldsymbol{Z}\boldsymbol{W}_o + \boldsymbol{b}_o$$

经可学习的 $\boldsymbol{W}_o$ 与偏置 $\boldsymbol{b}_o$ 融合各头的输出。

依据 《大语言模型：训练、精调与对齐》式 (2-33)–(2-38)；代码 2（2.5.2 节）

代码 2　类的构造与张量形状变换

```
1  class MultiHeadAttention(nn.Module):
2      def __init__(self, dim, heads):
3          super(MultiHeadAttention, self).__init__()
4          self.heads = heads
5          self.dim = dim
6          self.WQ = nn.Linear(dim, dim)
7          self.WK = nn.Linear(dim, dim)
8          self.WV = nn.Linear(dim, dim)
9          self.WO = nn.Linear(dim, dim)
10
11     def forward(self, X, mask):
12         B, L, D = X.size()
13         head_dim = self.dim // self.heads  # 单头词向量维度
14         Q, K, V = self.WQ(X), self.WK(X), self.WV(X)
15         Q = Q.reshape(B, L, self.heads, head_dim).transpose(1, 2)
16         K = K.reshape(B, L, self.heads, head_dim).transpose(1, 2)
17         V = V.reshape(B, L, self.heads, head_dim).transpose(1, 2)
```

- `__init__` 里建立四个线性层，输入输出维度都是 `dim`：前三层做投影，最后一层融合各头。
- 第 13 行用整除算出每头维度；`heads` 不能整除 `dim` 时会发生截断，所以实现上要求能整除。
- 第 15–17 行的 `reshape` 把特征维拆成「头数 × 每头维度」，`transpose(1, 2)` 再把头维提到序列长度之前。
- 变换后张量为 `(B, heads, L, head_dim)`，每个头在最后两维上独立做矩阵运算。

依据 《大语言模型：训练、精调与对齐》代码 2（2.5.2 节）

代码 2　注意力计算、拼接与测试

```
18      # 缩放点积注意力计算
19      Z = ScaledDotProductAttention(Q, K, V, mask)
20      Z = Z.transpose(1, 2).reshape(B, L, self.dim)
21      O = self.WO(Z)
22      return O
23
24  bs = 4
25  heads = 8
26  seq_len = 6
27  dim = 512
28  MHA = MultiHeadAttention(dim, heads)
29  dummy_X = torch.randn(bs, seq_len, dim)
30  dummy_mask = torch.zeros(1, 1, seq_len, seq_len, dtype=torch.bool)
31  out = MHA(dummy_X, dummy_mask)
```

- 输入 X 形状为 (4, 6, 512)；切分后 Q、K、V 形状为 (4, 8, 6, 64)，即 8 个头、每头 64 维。
- 掩码形状 (1, 1, 6, 6)，全为 False，不屏蔽任何位置。
- 输出 O 的形状为 (4, 6, 512)，与输入一致，可直接接入后续层。

依据 《大语言模型：训练、精调与对齐》代码 2（2.5.2 节）

2.6 小结

回顾本章要点，并在附录 2.6.1 中完整推导位置编码对注意力的必要性。

本节内容

本章要点

2.6.1　附录：位置编码的必要性

本章要点

模型结构

注意力是核心组件

编码器自注意力、解码器掩码自注意力与编码器-解码器注意力的核心计算相同，区别主要在掩码的构造。

训练

训练并行

教师强制加掩码，使单次前向即可预测所有位置，显著提高训练效率。

推理

推理串行

需要逐词生成直至 <EOS>，每步重算已生成词元，存在冗余。

复杂度与位置

二次增长与位置编码

自注意力的时间与空间复杂度随长度二次增长；位置编码使注意力感知词序（附录）。

延伸 GPT 与 Llama 是两种代表性的纯解码器架构，分别见第 3、4 章；高效训练与推理见第 9、10 章。

不加位置编码时的注意力分数

$$S_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^T \tag{2-69}$$

注意力分数是第 i 个查询与第 j 个键的点积，$\boldsymbol{Q}_i$、$\boldsymbol{K}_j$ 分别取 $\boldsymbol{Q}$、$\boldsymbol{K}$ 的第 i、j 行。

$$S_{ij} = (\boldsymbol{x}_i \boldsymbol{W}_q + \boldsymbol{b}_q)(\boldsymbol{x}_j \boldsymbol{W}_k + \boldsymbol{b}_k)^T \tag{2-70}$$

代入逐行投影 $\boldsymbol{Q}_i = \boldsymbol{x}_i \boldsymbol{W}_q + \boldsymbol{b}_q$、$\boldsymbol{K}_j = \boldsymbol{x}_j \boldsymbol{W}_k + \boldsymbol{b}_k$（式 (2-18)–(2-20) 的一行）。

$$S_{ij} = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T + \cdots \tag{2-71}$$

按分配律展开：$(\boldsymbol{x}_i \boldsymbol{W}_q)(\boldsymbol{x}_j \boldsymbol{W}_k)^T = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T$ 为第一项；其余含偏置的三项以 $\cdots$ 表示。

$$S_{ij} = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T \tag{2-72}$$

为简化推导，暂时忽略偏置项 $\boldsymbol{b}_q$、$\boldsymbol{b}_k$，只保留核心项。

依据 《大语言模型：训练、精调与对齐》第 2.6.1 节（附录）

置换不变性：词序被打乱时输出不变

$$\boldsymbol{O}_i = \sum_{j=1}^{n_k} S_{ij} \boldsymbol{V}_j$$

式 (2-73) 输出 $\boldsymbol{O}_i$ 是各值向量 $\boldsymbol{V}_j$ 以 S_{ij} 为权重的加权和（此处忽略缩放与 Softmax）。

$$\boldsymbol{O}_i = \sum_{j=1}^{n_k} S_{ij} \boldsymbol{V}_j = \sum_{j'=1}^{n_k} S_{ij'} \boldsymbol{V}_{j'}$$

式 (2-74) 若同一词元 $\boldsymbol{x}_j = \boldsymbol{x}_{j'}$ 换到位置 j'：S_{ij} 只依赖 $\boldsymbol{x}_i$、$\boldsymbol{x}_j$，$\boldsymbol{V}_j$ 只依赖 $\boldsymbol{x}_j$，每一项都不变；求和只改变顺序，所以 $\boldsymbol{O}_i$ 保持不变。

结论 只有注意力、没有位置信息时，打乱词序不改变输出，模型无法区分“大模型”与“模大型”。

依据 《大语言模型：训练、精调与对齐》第 2.6.1 节（式 (2-73)、(2-74)）

打乱词序：无位置编码不变，有位置编码改变

同一组三个词元，只把“大”与“模”的位置对调。左：分数只由词向量本身决定，输出逐项相同；右：位置向量 $\boldsymbol{p}_i$ 已随词元进入分数，对调后 $\boldsymbol{S}$ 不同，输出随之改变。

不加位置编码

分数只由词向量本身决定，与位置无关

序列 ①

大	模	型

序列 ② 交换“大”与“模”

模	大	型

输出 ①

0.55	0.75	0.62

输出 ②

0.55	0.75	0.62

打乱词序，输出逐项相同

加入位置编码

位置向量随词元一起进入分数，换位后不再相同

序列 ①

大 p_1	模 p_2	型 p_3

序列 ② 交换“大”与“模”

模 p_1	大 p_2	型 p_3

输出 ①

0.55	0.75	0.62

输出 ②

0.68	0.84	0.48

打乱词序，输出随之改变

加入位置编码后的注意力分数

$$S_{ij} = \boldsymbol{Q}_i \boldsymbol{K}_j^T \tag{2-75}$$

输入换成加入位置编码后的 $\boldsymbol{x}_i + \boldsymbol{p}_i$（见 2.3.3 节输入层），$\boldsymbol{Q}_i$、$\boldsymbol{K}_j$ 的定义不变。

$$S_{ij} \approx (\boldsymbol{x}_i + \boldsymbol{p}_i)\boldsymbol{W}_q \boldsymbol{W}_k^T (\boldsymbol{x}_j + \boldsymbol{p}_j)^T \tag{2-77}$$

忽略偏置项，与式 (2-72) 的简化相同。

$$S_{ij} = ((\boldsymbol{x}_i + \boldsymbol{p}_i)\boldsymbol{W}_q + \boldsymbol{b}_q)((\boldsymbol{x}_j + \boldsymbol{p}_j)\boldsymbol{W}_k + \boldsymbol{b}_k)^T \tag{2-76}$$

把 $\boldsymbol{x}_i$ 替换为 $\boldsymbol{x}_i + \boldsymbol{p}_i$，把 $\boldsymbol{x}_j$ 替换为 $\boldsymbol{x}_j + \boldsymbol{p}_j$。

位置项进入注意力分数

$$S_{ij} = \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T + \boldsymbol{x}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{p}_j^T + \boldsymbol{p}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{x}_j^T + \boldsymbol{p}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{p}_j^T$$

式 (2-78) 展开为四项：词与词、词对位置、位置对词、位置与位置。其中 $\boldsymbol{p}_i \boldsymbol{W}_q \boldsymbol{W}_k^T \boldsymbol{p}_j^T$ 同时涉及两个位置。

$$\boldsymbol{O}_i = \sum_{j=1}^{n_k} S_{ij} \boldsymbol{V}_j \neq \sum_{j=1}^{n_k} S_{ij'} \boldsymbol{V}_{j'}$$

式 (2-79) 当 $\boldsymbol{p}_j \neq \boldsymbol{p}_{j'}$ 时，$\boldsymbol{K}_j \neq \boldsymbol{K}_{j'}$，$S_{ij} \neq S_{ij'}$，输出随之改变。例如 (大, 模, 型) 与 (模, 大, 型) 中的“型”，加入位置编码后输出不同。

依据 《大语言模型：训练、精调与对齐》第 2.6.1 节（式 (2-78)、(2-79)）

位置编码点积的推导（一）：定义与展开

设位置 i、j 的二维绝对位置编码 $\boldsymbol{p}_i, \boldsymbol{p}_j \in \mathbb{R}^2$，$\theta_k$ 表示完整位置编码的第 k 个角度。

$$\boldsymbol{p}_i = (\sin(i\theta_k),\ \cos(i\theta_k))$$
$$\boldsymbol{p}_j = (\sin(j\theta_k),\ \cos(j\theta_k))$$

第 1 步　写出两个编码的分量：第一分量为正弦，第二分量为余弦。

$$\begin{aligned}\boldsymbol{p}_i\boldsymbol{p}_j^T &= \sin(i\theta_k)\sin(j\theta_k) \\ &\quad + \cos(i\theta_k)\cos(j\theta_k)\end{aligned} \tag{2-80}$$

第 2 步　按点积定义对应分量相乘再相加：第一分量得 $\sin(i\theta_k)\sin(j\theta_k)$，第二分量得 $\cos(i\theta_k)\cos(j\theta_k)$。

这一结果含有两个角度 $i\theta_k$、$j\theta_k$，还不能直接看出它只依赖 $j-i$，需要用三角公式化简（下页）。

位置编码点积的推导（二）：化为 cos((j−i)θ)

$$\cos(A-B)=\cos A\cos B+\sin A\sin B$$

第 3 步　两角差的余弦公式（三角恒等式）。

$$\begin{aligned}\boldsymbol{p}_i\boldsymbol{p}_j^{T}&=\sin(i\theta_k)\sin(j\theta_k)\\&\quad+\cos(i\theta_k)\cos(j\theta_k)\\&=\cos\big((j-i)\theta_k\big)\end{aligned}\tag{2-81}$$

第 5 步　由第 4 步的结果，位置编码的点积只取决于差值 $j-i$。

$$\begin{aligned}\cos\big((j-i)\theta_k\big)&=\cos(j\theta_k-i\theta_k)\\&=\cos(j\theta_k)\cos(i\theta_k)\\&\quad+\sin(j\theta_k)\sin(i\theta_k)\end{aligned}$$

第 4 步　令 $A=j\theta_k$、$B=i\theta_k$，则 $(j-i)\theta_k=A-B$。右端两项与式 (2-80) 的两项相同，只是乘法顺序不同。

依据　《大语言模型：训练、精调与对齐》第 2.6.1 节（式 (2-80)、(2-81)）

只依赖相对位置 $j-i$

- 式 (2-81) 的结果只取决于差值 $j-i$，与 i、j 各自的绝对位置无关。
- 对完整位置编码，每个角度 θ_k 的分量对都给出 $\cos((j-i)\theta_k)$，求和后仍只依赖 $j-i$。
- 位置编码把“相对距离”直接写进注意力分数，打破了置换不变性。
- 正文末段指出：在纯解码器结构中，位置编码对长度泛化未必绝对关键（NoPE）。

示例：取 $\theta_k = 0.5$

位置对 (i, j)	$j-i$	$\cos((j-i)\theta_k)$
(1, 2)	1	0.878
(3, 4)	1	0.878
(1, 3)	2	0.540
(2, 4)	2	0.540

> **注** 式 (2-81) 针对不加投影矩阵的点积。注意力中还有 $\boldsymbol{W}_q\boldsymbol{W}_k^T$，此时分数不再严格只依赖 $j-i$，但位置信息已显式进入分数。

文献 Kazemnejad et al., *The Impact of Positional Encoding on Length Generalization in Transformers*, arXiv 2023（NoPE）

思考题与实践题

1 自注意力的时间与空间复杂度各是多少？当序列长度从 n 翻倍到 $2n$ 时，两者分别变成原来的几倍？

2 表 2.3 中 s_{ij}（$i < j$）为何置为 $-\infty$？这对 $\boldsymbol{q}_i$ 有什么影响？

3 如何把 Transformer 去掉编码器，直接用于双语翻译任务的学习？解码器的输入与注意力层要相应做哪些改动？

4 生成 m 个词元需要几次解码器前向计算？与训练相比，慢在哪里？

5 掩码自注意力里，查询 $\boldsymbol{q}_i$ 究竟在检索什么？为什么它的检索范围必须截断到位置 i？

6 位置编码点积只依赖 $j - i$：相对距离越大，注意力分数会呈现什么特点？这与“远距离依赖更难学”是什么关系？

7 实践：用第 2.5 节的代码实现一个双语翻译 Transformer，跑通“编码器 + 解码器 + 输出层”的完整前向与训练。

8 实践：推导层归一化的梯度（对 $\boldsymbol{x}_i$、$\boldsymbol{\gamma}$、$\boldsymbol{\beta}$），并实现一个可反向传播的 LayerNorm 类。